Deusto

Facultad de Ingeniería
Ingeniaritza Fakultatea

Grado en Ciencia de Datos e Inteligencia Artificial

Datuen Zientzia eta Adimen Artifizialeko gradua

Proyecto fin de grado

Gradu amaierako proiektua

Acknowledgments

Abstract

This project proposes the development and implementation of a Multi-Agent system integrated with Vision Language Models (VLMs) for the execution of cooperative robotic tasks within a simulated NVIDIA Isaac Sim environment.

One of the major challenges lies in translating commands from a natural language format into precise robotic actions. To address this, the reasoning capabilities of the VLMs have been exploited for complex instruction interpretation and decision-making. A controlled environment where robotic agents must cooperate to manipulate and respond to objects in order to achieve a shared goal has been designed using NVIDIA's Isaac Sim framework. The integration of vision language models has enhanced generalization and promoted a more cooperative behavior of the agents against variations in the scene. The performance and learning efficiency has been evaluated in addition to metrics such as the transferable capacities of the models. Additionally, the project includes an in-depth analysis of state-of-the-art (SOTA) models, providing a comprehensive review of the current methodologies and core concepts that form the theoretical foundation of the proposed architecture, presented in the documentation.

The expected outcome is a demonstration and reflection of the empowerment of multi-modal models within robotic environments, which can be applied to real-world problems.

Index terms

Vision language models, multi-agent system, computer vision

Contents

List of Figures

List of Pseudocodes

List of Tables

1. INTRODUCTION

1.1 PROBLEM CONTEXT

The gap between human and robots still remains significant and leads to communication issues in robotic environments, the interpretation of commands may well be too rigid and may not translate as intended to the real scenario. Furthermore the task of coordinating multiple agents in an environment arises more issues including the previous, as the difficulty of synchronizing more than one agent raises exponentially due to integration and coordination issues. In order to tackle that gap, several techniques have been developed including simulation environments to reduce the *sim2real* hole of robotics deployment, however, there are still some performance issues these frameworks cannot cover. Reason why different artificial intelligence methodologies have started to been applied for robotics. In parallel, the rise of multimodal models including the vision language models (VLMs) has opened a new possibility frontier for bridging this gap. VLMs offer the ability of perceiving visual scenes alongside natural language reasoning simultaneously, extending a more flexible interface for human-robot interaction. Nevertheless, the successful integration of this technology to robot physical execution of multiple tasks remains an open challenge to be explored throughout this project.

1.2 MOTIVATION

Vision language models had little to no other integration in other field rather than *chatbot* plugins. Reason why I wanted to experiment VLM integrations in another such interesting field as robotics. I also wanted to experiment with a high-level reasoning engine able to think for itself instead of relying on hard-coded actions and positions to give to the agents. Likewise, this project encouraged to use state of the art models and frameworks for the experimentation part.

1.3 OBJECTIVES

1.3.1 Main objective

Experiment with the integration of reasoning capacities of a vision language model in a controlled simulated multi-agent robotic environment to solve cooperative manipulation tasks while covering the whole process alongside the SOTA of the models in the documentation.

1.3.2 Specific objectives

- Design progressively more complex environments in the robotic scenario simulator.
- Implement a communication system between VLM and the agents.
- Define metrics to measure the success rate of cooperative tasks, the response time to natural language commands, and the model's ability to generalize across different environmental variations.
- Evaluate the robustness of the learned policies and the VLM's perception under different levels of domain randomization to assess the feasibility of future deployment on physical hardware.

1.4 PROJECT SCOPE

What the project includes:

- The practical area of the project is strictly developed in NVIDIA IsaacSim, taking advantage of its complex physics engine and realistic rendering capacities (RTX).
- Vision language models integration as high-level decision-making orchestrators.
- Communication pipeline orchestration.

What is not covered in the project:

- Physical display of the robots, they are strictly displayed in the controlled environment.
- Foundational VLMs training from scratch. The used model(s) are pre-trained open source alternatives.

2. STATE OF THE ART

2.1 EVOLUTION OF MULTIMODAL LEARNING

Multimodal learning is a type of *deep learning* that processes multiple types of data referred as modalities [5]. That data may belong to different domain such as text, audio, images or video. Multimodal learning was proposed at the beginning of the DL period, around year 2011, whilst have raised their popularity when they were mixed with large multimodal languages in the recent years.

2.1.1 Multimodal deep Boltzmann machines

Boltzmann machines are known to be the multimodal data dealing algorithm ancestors. Invented in 1985 and named after the *Boltzmann distribution*, these machines can be seen as stochastic neural networks. Multimodal deep Boltzmann machines can process and learn from different types of information simultaneously, having a separate machine for each modality. Its units are divided into visible and hidden units, units which represent neurons with a binary output indicating whether they are activated or not.

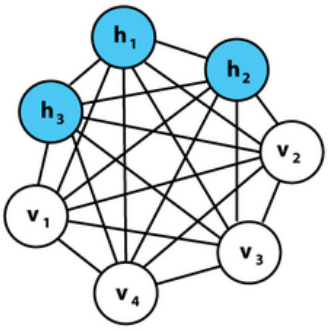

Figure 2.1: Deep Boltzmann machine.

However, learning is impractical using these machines because of the exponential computational time and not optimized architecture. Successor *restricted Boltzmann machine* alternative was designed to deal with efficiency by just allowing connections between hidden units and visible units. Lowering the computational power and making training more efficient.

2.1.2 Multimodal Transformers

Transformers can also be adapter for modalities, usually finding a way to *tokenize* the data domain. Even if multimodal transformers can be trained from scratch, a 2022 *transfer learning* study showed that fine-tuned transformers originally pretrained with natural language data can show competitiveness in logical and visual tasks.

2.1.2.1 The Perceiver

Perceivers, are a variant of the transformer architecture exclusively adapted for processing multimodal data. The Perceiver is designed as a general architecture that can learn simultaneously from large amounts of heterogeneous data, implementing *asymmetric attention* mechanisms. Asymmetric attention consists of architectural modifications that break the "standard symmetry" to address specific domain constraints. This approach leads to outperforming specialized models on classification tasks. It iteratively alternates cross-attention and latent self-attention blocks to project a high-dimensional input byte array to a fixed-dimensional latent bottleneck before processing it using a deep stack of transformer-style self-attention blocks in the latent space [14].

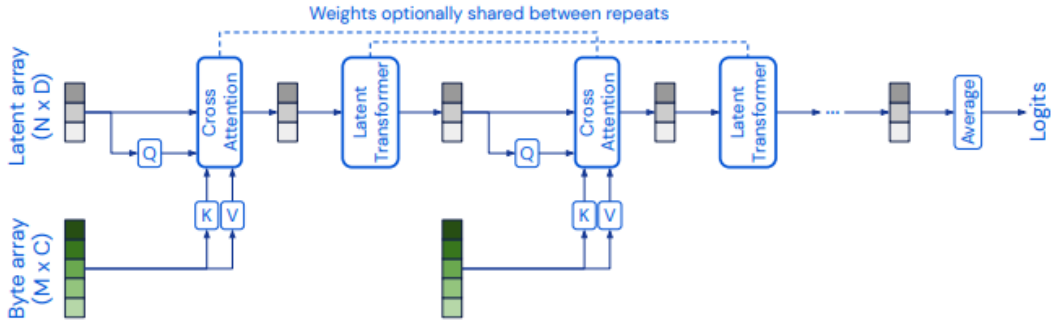

Figure 2.2: Perceiver architecture from the referenced paper

2.1.2.2 Image generation Transformer models

- **DALL-E 1** (2021): When it comes to image generation models, *DALL-E 1* is not a *difussion model* (latent variable generative models). It uses a decoder-only transformer which autoregressively generates a text followed by the token representation of an image, which is ten converted by a variational autoencoder to an image.

- **Parti** (2022): On the other hand, models like *Parti* use an encoder-decoder schema. Where the encoder processes a text prompt and the decoder generates the token representation of an image.

- **Muse** (2023): uses an encoder-only architecture trained to predict the masked image tokens from unmasked image tokens. In training, all input tokens are masked, and the highest confidence predictions are revealed

- **Phenaki** (2023): is a text-to-video-model which uses a bidirectional masked transformer conditioned on pre-computed text tokens. Then, the tokens are decoded to a video.

2.1.3 Applications of Multimodal Learning

The integration of data from different domains has allowed a deeper understanding of models for different type of tasks requiring multimodality:

- **Cross-modal retrieval**: is a subfield of information retrieval which enables users to search and retrieve information across different data modalities [34]. Differing from the traditional retrieval systems, cross-modal retrieval bridges different types of media to facilitate flexible information access. There are several combinations of retrieval depending on both the input and output data domain, e.g. *audio-to-video, video-to-text, ...*

- **Content generation**: models such as *DALL-E* generate images from textual descriptions.

- **Classification and missing data retrieval**: multimodal *Deep Boltzmann Machines* outperform traditional shallow machine learning algorithms like the *support vector machines* in classification tasks and are able to predict missing instances in multimodal datasets.

- **Robotics and human-computer interaction**: multimodal learning has improved the interaction in robotics and AI by integrating sensory inputs like speech, vision and touch, aiding autonomous systems and human-computer interaction.

2.2 ADVANCEMENTS OF VISION LANGUAGE MODELS

While the multimodal models mentioned before established the standard workflow for processing heterogeneous data, they often treated modalities independently to be combined or translated. The next step in the evolution are the *Vision Language Models*, representing a shift towards a more reasoning approach

rather than bare multimodal output goals. Instead of just generating an image from text or retrieving text from a video, VLMs think across all the domains simultaneously, implementing the reasoning capabilities from *large language models* to interpret visual scenes.

VLMs are the result of the evolution to earlier image captioning systems, systems which were designed primarily to take isolated images and produce descriptions. However, this systems just received the image as input, not an image-text pair input as the vision language models do now [40]. I will now go over some of the huge advancements that resulted when transitioning from the traditional multimodal learning to these new vision language models and their main differences:

- **Different input handling**: while with multimodal learning you would have a separate model for handling data from a specific domain e.g. one for dealing with text and another for video, VLMs were designed to receive input pairs of data from different modalities. Usually being (image, text) pairs. The VLM is able to align concepts belonging to different modalities inside a single shared vector space.

- **Change of objective**: multimodal learning mainly covered traduction problems from one domain to another, whereas modern VLMs seek inter-domain reasoning. The capability to reason and distinguish the same concept in different modalities.

- **Representation in the latent space**: vision language models try to preserve the same mathematic meaning for the same concept across all domains.

2.2.1 Earliest models

The early image captioning systems from around year 2010 used the *encoder-decoder* architecture, which will be described later on. These models would encode the patched images and vectorize them into feature vectors, which were then fed to the decoder part in order to generate the associated output description. When it comes to the strategies implemented by this earliest models include combination of handcrafted *visual features* to encode the image patches and *n-gram* templates to generate the description.

Those visual features can refer to any visual property like points, edges or even objects in the image.

On the other hand, n-grams are statistical language models that calculate the probability of the next word in a sequence from a fixed size window of previous words. Depending on the number of previous words considered, we can get bigram models if just one previous word is considered, if it's two is a trigram

model, ... up to n - 1 considered as *n-gram* models and finally an unigram if no previous context is considered.

The general approximation for a sequence of words $w_1, \ldots, w_m$ is:

$$P(w_1, \ldots, w_m) \approx \prod_{i=1}^{m} P(w_i \mid w_{i-(n-1)}, \ldots, w_{i-1})$$

- Unigram Model ($n = 1$): No context, independent probabilities.

$$P(w_1, \ldots, w_m) \approx \prod_{i=1}^{m} P(w_i)$$

- Bigram Model ($n = 2$): Depends on the single previous word.

$$P(w_1, \ldots, w_m) \approx \prod_{i=1}^{m} P(w_i \mid w_{i-1})$$

- Trigram Model ($n = 3$): Depends on the two previous words.

$$P(w_1, \ldots, w_m) \approx \prod_{i=1}^{m} P(w_i \mid w_{i-2}, w_{i-1})$$

2.2.2 Deep learning based models

When the deep learning neural networks arose and became dominant near year 2015, newer models and strategies also emerged. Convolutional neural networks (CNNs) started to being used for image encoding and recurrent neural networks (RNNs) to generate the captions.

Despite their respective success in independent tasks, the combination of both models hit a plateau when dealing with VLM scenarios due to their infrastructure limitations. Convolutional neural networks work great when capturing local spatial patterns but they struggle with global long-term dependencies of the whole image without a large number of layers. On the other hand these *seq2seq* models performed poorly with long-term dependencies due to their hard-coded sequentiality. Often leading to the model forgetting the processes carried in the first layers of the architecture.

2.2.3 The Transformers architecture standard

The transition from sequential models combined with convolutional neural networks to *Transformers* solved the long-term dependency capturing issues and lack of global visual feature captioning. Still using these models, VLMs are able to attend to the whole input prompt simultaneously thanks to parallelization and attention mechanisms and also due to the specific vision transformer architectures.

The functionality of the basic Transformers architecture can be wrapped into three main stages:

1. The encoders transform the input sequence into high-dimensional numerical representations called *embeddings* which capture all the semantic and positional meaning of the tokens belonging to the input. In order to deal with those high-dimensional embeddings in the next steps of the architecture where the order of the input sentence words cannot be distinguished as the transformer processes all tokens in parallel, the *positional encoding* was implemented. These encodings typically use sine and cosine functions of different frequencies to provide a unique signature for each position:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

   This ensures that the model captures the structural context (e.g., distinguishing between "dog bites man" and "man bites dog") before any attention is calculated.

2. Then they use a mechanism called *self-attention* which allows the encoder to focus on the most relevant tokens of the sequence independent of their position. This is where the magic of the transformers actually happens. The implementation of the attention mechanism replaced the use of previous *seq2seq* models which relied on recurrence. The original paper also introduced the concept of *scaled dot-product attention*:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

   where respectively Q, K, V are the query, key and value matrices, and d_k the dimensionality value. This approach eliminates the need for RNNs, ensuring parallelizability for the architecture. In the self-attention mechanism, those matrices are dynamically generated for each input sequence, allowing the model to focus on the different parts of the input sequence at different timesteps. *Multi-*

head attention enhances the process by introducing several parallel attention heads simultaneously, enabling each head to learn different Q, K and V matrix linear projections and focusing on multiple aspects instead of a single one. MHA ensures that the input embeddings are updated from a more varied and diverse set of perspectives, and after the attention outputs are calculated, they are concatenated and passed through a linear layer transformation to generate the output.

3. Finally the decoders of the transformers take both the output of the self-attention mechanism and the encoder generated embeddings to generate the most statistically likely output sequence. In the case of VLMs, it can also use those attention-weighted embeddings to output the most loyal visual representation compression.

2.2.3.1 VLM tasks with Transformers

The transformer architecture replaced the sequential recurrent neural networks in the role as decoders. In parallel, the network parameter training started relying on image-text pair datasets, and the scope of VLMs widened until reaching other tasks.

- **Phrase grounding**: Among those tasks, *phrase grounding* could be mentioned. Task involving both natural language processing and computer vision where words or phrases within a sentence are associated to corresponding patches in an image [30]. Consists of identifying the spatial location within an image that aligns with the meaning of a given phrase, enabling models to map visual and textual data.

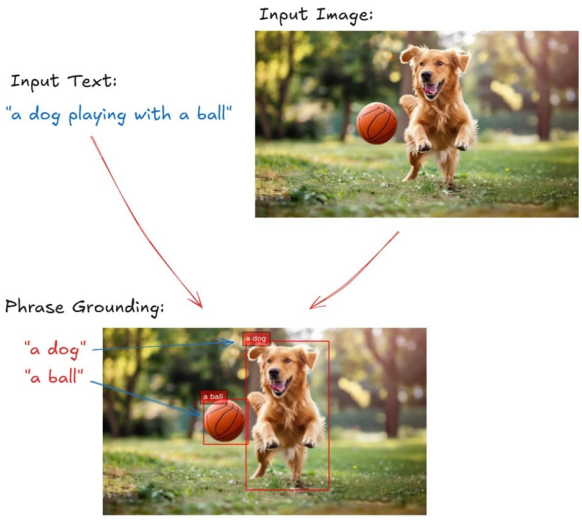

Figure 2.3: Graphical example of phrase grounding.

- **Visual Question Answering** (VQA): Another task which still remains as perhaps the most relevant one is *visual question answering*. Specific task where the model is provided with an image and a specific related question, and it must generate an accurate response to it based on the visual content. Being its core pillars the object detection duty for identifying the objects in the image, then scene understanding to map the relations between objects and then finally text comprehension for embedded text interpretation within an image [11].

However, modern VLMs do not follow the conventional 2017 paper [31] workflow, they typically use different transformers configuration. Some models use only the encoder part of the architecture for both modalities, completely ignoring the traditional encoder component. While other recent models use the encoder for feature extraction and use an independent encoder for language modeling. Different approaches that will be explained in the following sections.

2.3 FROM ENCODERS TO GENERATIVE LLMS

2.3.1 Language encoder

In traditional multimodal systems, the language encoder functioned as a bidirectional processor, having the role of generating contextualized latent representation of the input where each token could attend to its surrounding tokens in both directions. However, in modern VLM architectures, the encoder has been linked to the LLM engine. Functioning as a preparation of the linguistic information to interact with the visual tokens within a shared vectorial space.

2.3.2 The LLM transition: from encoders to decoder-only architectures

The most significant paradigm shift in VLM evolution is the transition from symmetric *encoder-decoder* systems to *decoder-only* architectures. This move toward generative LLMs is driven by three fundamental technical advantages:

- **Causal Attention vs. Bidirectional Attention**: Unlike encoders that allow tokens to see the entire sequence at once, decoders implement a *causal mask*. This ensures that during training and inference, each token can only attend to itself and preceding tokens. This directional dependency is what enables autoregressive generation, the ability to reason and predict the next word in a conversation.The causal mask M is typically a lower triangular matrix, where the attention score

between position i and j is set to $-\infty$ if $j > i$:

$$M_{ij} = \begin{cases} 0 & \text{if } i \geq j \\ -\infty & \text{if } i < j \end{cases}$$

After applying the *softmax* activation function, these $-\infty$ values become zero probability, effectively blocking information flow from future tokens.

- **In-Context Learning and Reasoning**: By utilizing a decoder-only system, the VLM treats the input image as a visual prefix. The model does not just classify the image, rather it predicts the logical continuation of a sequence $[Image, Instruction, Response]$. This allows the model to influence *Chain-of-Thought* (CoT) reasoning, where it can articulate its thought process before arriving at a final answer.

- **Scalability and Unified Training**: Decoder-only models are simpler to scale because they consolidate the entire reasoning process into a single set of weights. In a model like *LLaVA*, the LLM receives a unified sequence of tokens: $\mathcal{S} = [v_1, \ldots, v_n, t_1, \ldots, t_m]$, where v represents aligned visual pseudo-tokens and t represents text tokens. The model treats these two modalities as a single language, significantly reducing the architectural bottleneck between "seeing" and "speaking."

2.4 VISION TRANSFORMERS

2.4.1 Vision encoder & Vision Transformers (ViT)

Compared to earlier VLMs which used *convolutional neural networks* (CNNs) for feature extraction such as the colors, shapes and textures from the input, modern ones employ an architecture called *visual transformer* (ViT).

Visual transformers process images into smaller patches (rather than text tokens) and treats them as sequences, implementing also the self-attention mechanism across the patches to create a transformer-based representation of the input message.

ViTs were designed as *convolutional neural network* alternatives in computer vision applications, having different biases, training stability and data efficiency. Compared to CNNs, vision transformers are less efficient yet they have higher capacity.

The image available below 2.4 references the original architecture published in the 2020 paper [9], being basically a *BERT*-like encoder-only transformer. It also uses special token `<CLS>` in the input side to refer to the start of the sentence token, and the output vector is used as the input of the final MLP head. Transformers, including the ViTs, measure the relationship between pairs of input tokens with the attention mechanism. In the case of images, the basic unit of analysis is the pixel. Vision transformers compute relationships between small regions of pixels (patches) and they maintain the spatial order with the positional embeddings computation. Each section of the architecture is passed through a linear sequence and multiplied by the embedding matrix, being the result with the position embedding fed to the transformer. Let's overview its functionality:

1. The input image is of type $\mathbb{R}^{H\times W\times C}$, where H, W, C are height, width, and channel (RGB). It is then split into square-shaped patches of type $\mathbb{R}^{P\times P\times C}$. For each of the patches, the respective "patch embedding" is obtained by pushing the patch through a linear operator and transformed through the *positional encoding* layer. Both embeddings are added and pushed to the various transformer encoder layers.

2. Inside those encoder layers, the attention mechanism incorporates more and more semantic relations to the image patch embeddings.

3. Depending on the end task desired, one could add different layers in the end steps. For instance, for a classification task like the one performed in the image, a shallow *multi-layer perceptron* (MLP) layer is added on top to output the probability distribution over the classes. As fact, the original paper goes for a linear-GeLU linear softmax network for classification.

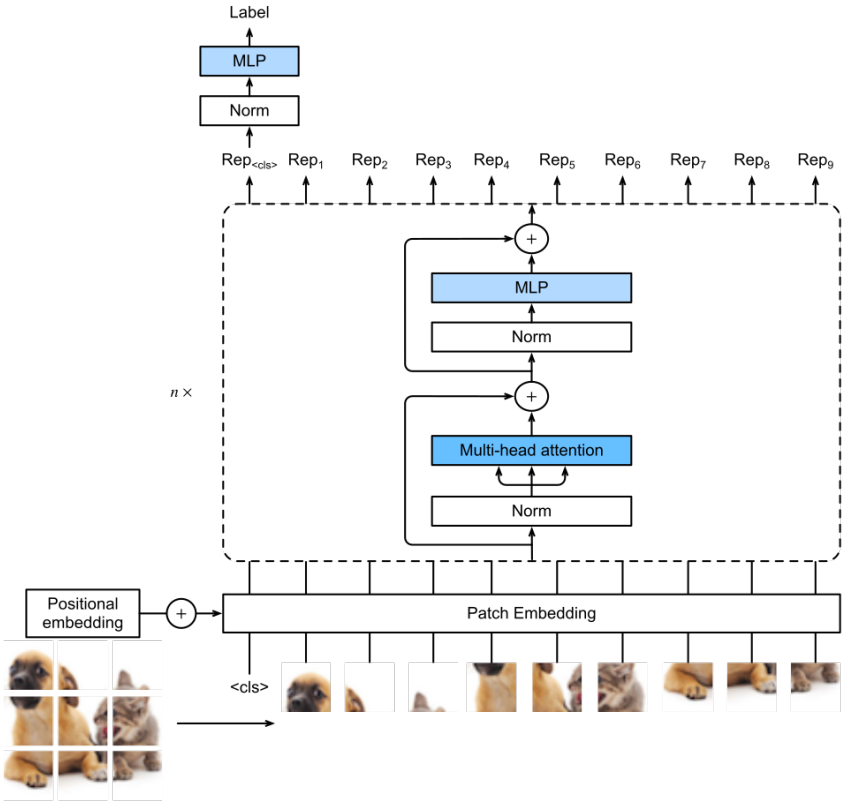

Figure 2.4: Vision transformer architecture

There are other ViT variants which have been published and reported having used different techniques to deal with image manipulation. The following are some of the architectural improvements:

- **Global average pooling (GAP)**: unlike the original paper where the `[CLS]` token is used to represent the beginning of the sequence emulating BERT, this approach does not use it. It simply takes the average of all output tokens as the final classification token. In the original report, it was mentioned to be equally efficient to the original approach.

- **Multi-head attention pooling (MAP)**: it applies a new *multi-head attention* block to pooling. It takes the output of the ViT this being the list of vectors $x_1, x_2, \ldots, x_n$, whereas its output is $\text{MultiheadedAttention}(Q, V, V)$. Where Q is the trainable query vector and V the matrix with the rows being the input vectors. More recent papers state that both GAP and MAP show a better performance than the basic BERT-like pooling.

- **Masked Autoencoder**: this architecture involves two vision transformers put end-to-end. The

first one takes the input image patches with positional encoding and output the vectors representing each patch (same encoder procedure seen until now). This is where the second component being the decoder (even if it still an encoder-only transformer) takes the first encoder's output with positional encoding too and outputs the image patches again. See the architecture in the image below 2.5.

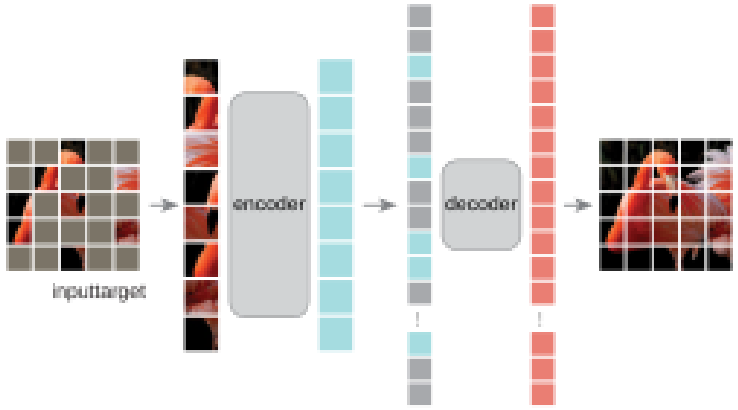

Figure 2.5: Masked autoencoder architecture

During the training process, a percentage of the splitted patches is masked by special mask tokens, while leaving others untouched (the sweet spot has been said is around 75% mask ratio). The whole network's task is to reconstruct the whole image with the missing "puzzle pieces" (being the masked patches). Masked patches are added back to the output of the encoder block as mask tokens and both are fed into the "decoder". A reconstruction loss is computed for the masked patches to assess network performance. In the prediction stage, the decoder is left over, the input image patches are processed and the resulting vector representation is fed to the encoder.

- **DINO (self-DIstillation with NO labels)**: like the masked autoencoder procedure, DINO is a self-supervised way to train a vision transformer. Acting as a "teacher-student" method, the student being the model itself and the teacher, an exponential average of the student's past states, must be able to transfer its knowledge to the smaller model [37]. The loss function it uses is the *cross-entropy loss* between the output of the teacher network and the output of the student version. The inputs of the networks are two different crops of the same image, represented as $T(x)$ and $T'(x)$, where x is the original image. And as mentioned before, the network of the teacher represents the exponential decay average of the student's past parameters:

$$\theta_{t'} = \alpha\theta_t + \alpha(1-\alpha)\theta_{t-1} + \dots \tag{2.1}$$

All put together, the loss function stands as followed:

$$\mathcal{L}(f_{\theta_{t'}}(T(x)), f_{\theta_t}(T'(x))) \tag{2.2}$$

In order to finish the ViT research subsection, I would like to make a little comparison with their "ancestor" model, the *convolutional neural networks* (CNNs).

The fundamental distinction between Vision Transformers and the convolutional neural networks lies in their inductive biases. CNNs are built on the principles of locality and translation invariance, using sliding kernels that assume pixels close to each other are more related than those far apart. This bias allows CNNs to be highly efficient with smaller datasets and provides a natural robustness to local perturbations and noise. In contrast, ViTs discard these spatial assumptions. By treating an image as a sequence of patches and applying a global self-attention mechanism, a ViT can capture long-range dependencies between distant parts of an image from the very first layer. This flexibility allows ViTs to surpass CNN performance on massive datasets where the model can "learn" the spatial relationships from scratch, rather than having them hard-coded into the architecture.

However, this architectural freedom comes with a significant trade-off in terms of optimization and data requirements. Because ViTs lack the "shortcuts" provided by the convolutional structure, they are notoriously sensitive to hyperparameter choices and the selection of optimizers, often requiring advanced strategie and specific learning rate schedules to achieve stability. While a CNN's computational complexity scales linearly with the number of pixels, the self-attention in ViTs is quadratically complex $O(N^2)$ relative to the number of patches N, making high-resolution processing more resource-intensive. Consequently, while CNNs remain the more time-efficient and stable choice for many standard applications, ViTs have become the preferred backbone for large-scale Vision-Language Models due to their superior ability to scale and their capacity for modeling the complex, global context required for multimodal understanding.

2.5 MULTIMODAL ALIGNMENT: CLIP

2.5.1 CLIP: Contrastive Language-Image Pretraining

In the year 2021, OpenAI launched the approach that would totally revolution the evolution of vision language models, they released the method of *contrastive language-image pretraining* also known as *CLIP*.

Technique for training an image-text pair into neural network models, having an exclusive neural network for image and text understanding purposes, adopting a contrastive objective.

When it comes to the algorithm itself, each neural network receives the input in the corresponding format and returns a single semantic/visual representative vector [36]. The objective to maximize by both models is the best alignment between both output vectors in the shared vector space, mapping the similar text-image pairs as close as possible while putting apart the most dissimilar ones. The training relies on a huge dataset with image-text pairs as and the two output vectors are considered similar depending on the *dot product* value.

$$s_{i,j} = \mathbf{v}_i \cdot \mathbf{w}_j = \sum_{k=1}^{d} v_{i,k} w_{j,k} \tag{2.3}$$

where $\mathbf{v}_i$ and $\mathbf{w}_j$ represent the embedding vectors generated by the text and image encoders respectively, and d denotes the dimensionality of the vector space. The loss function used is the multi-class N-pair loss, which is a symmetric *cross-entropy loss* over the similarity scores. The function fosters the dot product between the matching image and text vectors to be as high as possible, while discouraging high dot products between mismatching pairs.

$$\mathcal{L} = -\frac{1}{N} \sum_{i} \ln \frac{e^{v_i \cdot w_i / \tau}}{\sum_j e^{v_i \cdot w_j / \tau}} - \frac{1}{N} \sum_{j} \ln \frac{e^{v_j \cdot w_j / \tau}}{\sum_i e^{v_i \cdot w_j / \tau}} \tag{2.4}$$

being parameter T the temperature parameter (T>0) which is parametrized and learned during the training. Other loss functions are possible, such as the *Sigmoid CLIP* function.

Once gone over the algorithm's properties, let's dive deep into the architecture:

- **Image model**: when it comes to the image encoder used in most of the CLIP versions, they all usually are some kind of *vision transformers* (ViT). Vision transformers often vary on the patch size, being that size the nxn dimensions of the patch the image was divided into. Alternatively, *convolutional neural networks* (CNNs) can be used as well (such as *ResNet*), the essential thing is to ensure the sizes of the image and text model output vectors are the same size.

- **Text model**: the text encoding models are usually *transformers*. Even if it is heavily inspired by the transformers report published by OpenAI [22] and shares similarities with the *GPT* architecture (decoder-only), the text model serves as a text encoder. Furthermore, its functionality is inspired by encoder only models such as *BERT*, while sharing the dimensionality and number of parameters

with decoder-only models as mentioned before.

After going over the properties of both encoders that make the whole CLIP model, I will detail the model's workflow inspired on the image below 2.6.

1. The image of the image-text pair input is first splitted into smaller sized images named patches and then fed into the vision encoder, which as mentioned earlier, can be any kind of architecture such as a vision transformer (ViT) or convolutional neural network (CNN). The output of the vision encoder will in fact be a vector representation of all the features that have been captured: shapes, texture, colors, objects, spatial relations, ... etc.

2. Now it's time the model turns those embeddings into real semantic representations, nevertheless, the model still does not know how to label concepts directly. So the model must know combine the embeddings obtained in the image model with the ones from the text model output, aligning them into a shared vector space establishing the real relations and links between the pairs. This bridging mechanism between the image and text pair (which will be described deeper in the corresponding section) is where the CLIP model's magic really shines. CLIP uses a contrastive alignment strategy in order to put correct pairs closer and dissimilar pairs further in the vector space.

3. In the standard CLIP workflow, the model does not generate new text; instead, it performs *zero-shot prediction* by calculating the similarity between the image embedding and a set of candidate text embeddings. It identifies which text snippet has the highest dot product with the image, effectively "selecting" the best label.

4. In more advanced generative extensions, these aligned visual features are passed to an LLM. The LLM treats the image embeddings as part of its input sequence, allowing it to generate complex descriptions or provide answers to specific questions in a task known as *Visual Question Answering* (VQA).

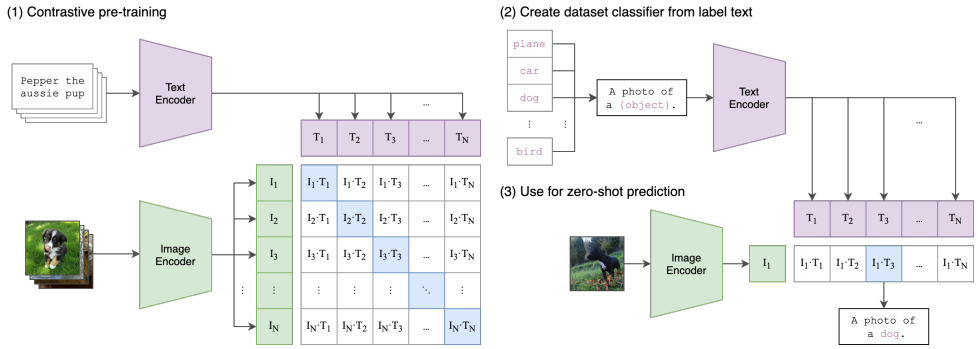

Figure 2.6: CLIP workflow

The reason why these models work so well is because of the millions of image-text pair instances used for training and due to the model's ability to learn and deal with statistical spatial correlations that are shown both in the images and texts.

The dataset the model was trained on is a *WebImageText* (WIT) containing more than 400 million pair instances scraped from the internet, almost half a million text queries and more than 20,000 image-text pairs as query. All of those queries were generated by the most frequent words of the english Wikipedia and then extended using bigrams. The dataset remains private and has never been released.

Last but not least, images are preprocessed by dividing them into the RGB (reed, green and blue) channels and normalizing the values so they fall between range 0 to 1. Finally those values are merged with the mean and standard deviation of the images from the training dataset. In case the input image does not have the desired resolution (224x224 usually) the image is scaled by *bicubic interpolation* [35] to the shorter sides is the same as the native resolution, and then the central square of the image is cropped out.

2.6 MODALITY BRIDGING AND PSEUDO-TOKENS

2.6.1 Modality bridging mechanism

The bridge briefly mentioned in previous sections, serves as the "mathematical handshake" between the visual and linguistic domains. Because the vision encoder and the LLM are often trained independently, their output dimensions and feature spaces rarely match. It's in those scenarios where the *dimensionality alignment* technique is applied. For example, an encoder such as ViT-L/14 may produce embeddings in

a 1024-dimensional space (1024 embedding size), whereas a Llama-based LLM requires 4096-dimensional inputs.

To deal with different size embeddings, several algorithms have been developed, being the *Manifold alignment* [38] one of them. This algorithm assumes that both the image and textual data sets exist in different high-dimensional spaces, still they share an underlying common semantic structure, also called *manifold*. The bridge's true purpose is to find a mapping that preserves the local geometric relationships of the two disparate sets. By minimizing the distance between related points while preserving the intrinsic structure of each domain. A great result would indicate the ability of the model to surpass the modality gap and project the same token close to each other across all modality spaces.

2.6.2 Pseudo-Tokens

Once the different embeddings are aligned, tokens are referred as *pseudo-tokens* or *visual tokens*. There is an interesting paper where this kind of tokens are analyzed more in depth, I am talking about the *A Sentence is Worth 128 Pseudo Tokens: A Semantic-Aware Contrastive Learning Framework for Sentence Embeddings* paper [28]. The study argues that a single sentence embedding is still not enough to capture visual complex meaning, it states that is necessary to use a fixed number of these new generated pseudo-tokens to actually represent a concept or image. In the context of VLMs, pseudo-tokens ensure that the LLM is able to look for different parts of the visual manifold as if they were a structure sequence of words, making easier to align between high-dimensional visual and semantic features.

When it comes to practical cases, this alignment is implemented through varying levels of architectural complexity. The simplest and most common approach, seen in models like LLaVA, utilizes a *linear projection* or a shallow *multi-layer perceptron* (MLP) to perform the manifold mapping. On the other hand, more advanced architectures like Flamingo or BLIP-2 use bottleneck structures such as the *perceiver resampler* or the *q-former*. These use a set of learned queries to attend to the encoder's output, effectively summarizing the visual space into a fixed, manageable number of pseudo-tokens. This prevents the LLM from being overwhelmed by the high-resolution raw output of the vision encoder, especially when dealing with high-resolution images or video sequences.

2.7 TRAINING STRATEGIES IN VLMS

Vision language models are trained not only in a single step process, but in a multi-stage training workflow, starting from the alignment of two disparate data modalities going to refining the model's ability to deal

with natural instructions. The methodologies can be categorized into paradigms:

2.7.1 Contrastive Pre-Training

: The core of contrastive methods is the idea of learning how to contrast between pairs of similar and dissimilar data points [23]. A pair of similar data points is called positive sample if both points are different representations of the same data instance, in our case multimodal representations of the same instance. Whereas the negative samples are those pairs in which the data points belong to different examples. While computer vision methods learn from input-input (image-image) pairs and natural language processing methods deal with input-output (text, label) pairs, vision language models receive input-input (text, image) pairs. Previously mentioned models such as *CLIP* [22], apply this learning approach to maximize the *cosine similarity* distance metric between matching image-text pairs in a shared embedding space, aiming to maximize the metric for the positive samples while minimizing the negative ones. The loss function used is the *cross-entropy* function.

2.7.2 Generative Pre-Training

While contrastive learning focuses on aligning data points from different modalities in a shared single embedding space, generative pre-training shapes that task as a sequence-to-sequence problem. Using thie *Prefix Language Modeling* (PrefixLM) approach [33], the model receives a prefix consisting of a pair of image patches and a text sequence. The goal of the model is to predict the continuation of that textual sequence, just like *seq2seq* models like *RNNs* and *LSTMs* try to predict. The loss function used is the negative log-likelihood loss, identical to the ones used in large language models like *GPT*:

$$\mathcal{L}_{PrefixLM} = -\sum_{t=1}^{|y|} \log P(y_t \mid y_{<t}, x_{prefix})$$

where x_{prefix} represents the combined image and text prefix and y_t is the text token at time t.

2.7.3 Visual Instruction Tuning

This strategy involves fine tuning the model with smaller datasets of higher quality, usually being (instruction-following) pairs where humans ask questions and the model replies. Supervised fine-tuning and parameter-efficient fine tuning methods are the most used ones, where only a small subset of the parameters is updated to avoid the LLM to forget its original knowledge.

2.7.4 Vision Language Models Training Paradigm

When it comes to the training process of VLMs there are two main strategies to be adopted, strategies which will be discussed and which are visible in the referenced image below 2.7. Those paradigms are no other than training a vision language from scratch versus starting from the base point of a pre-trained large language model as a frozen backbone. In the following section the procedures, differences and when to use each case will be studied.

2.7.4.1 Training a VLM from Scratch

Training the whole vision vision language model from zero differs significantly from starting from the base of having a pre-trained LLM. Usually a different training procedure and methodologies are applied, methodologies as the ones mentioned still *contrastive pre-training* is the most used approach combined with *self-supervised learning* techniques [1]. The training procedure would be the ordinary one, showed in the upper side of schema on the referenced image. Consisting on training both encoders, getting the respective vectors and applying contrastive alignment to form the final shared vector space.

2.7.4.2 Starting from a pre-trained LLM

On the other hand, starting from the checkpoint of already having a pre-trained large language model reduces or at least modifies the conventional training workflow. The text embeddings taken from the text encoding would be directly fed into the LLM without any previous alignment, however, there are two different alternatives to tackle the alignment task in for this training workflow variant:

1. **The Projector**: maps visual features extracted from the vision encoder with the text embeddings returned from the large language model, mapping which usually consists of multilayer perceptron (MLP) layers responsible for transforming high-dimensional visual representations into compact embedding tokens to be aligned with textual modality. The projector can be trained alongside the rest of the model or freezing the LLM in order to preserve the pre-trained knowledge. Among the modern models which follow this paradigm we have *LLaVa, Qwen-VL* and *Nvidia VLM*.

2. **Optimization Strategies (Joint vs Freeze Training)**: the second task involves updating a portion of the weights while freezing the others. *Joint training* refers to updating the weights of all components of the model in parallel without freezing any (including the weights of the LLM and projector layers), approach taken by models such as *Flamingo*. *Freeze training* on the other

hand, selectively freezes some components of the model during training, with the aim of preserving knowledge obtained prior to the training to be adapted to new tasks. The most common strategies are freezing the pre-trained vision encoders while fine-tuning the projector layers, gradually "unfreezing" the frozen parameters.

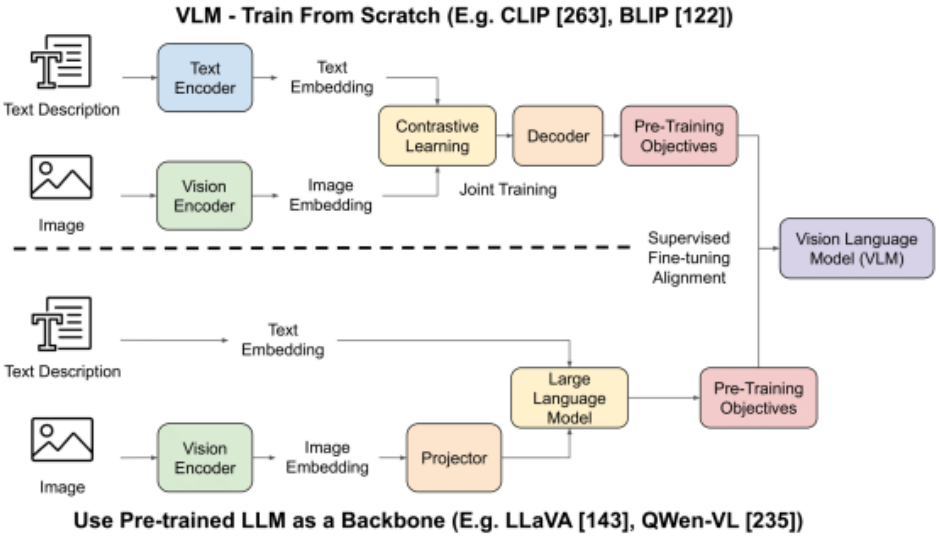

Figure 2.7: VLM trained from scratch vs training strategy using a pre-trained LLM as backbone

2.8 CURRENT SOTA VLMS

The current vision language models are not judged exclusively by their ability to align image-text pairs, but by their system level efficiency, long context reasoning and architecture. This section will analyze the leading *SOTA* models of the last years, models which represent the various approaches to the modality gap. Furthermore, these models have started to be implemented in chatbots to support multimodality user interaction.

2.8.1 State of the Art Models

Following the same structure redacted in paper *A Survey of State of the Art Large Vision Language Models: Alignment, Benchmark, Evaluations and Challenges* [1], some of the key models will be listed in a table form, alongside the metrics used to compare the models used in the paper.

Model	Year	Architecture	Training Data	Params	Vision Encoder	Backbone
Flamingo	2022	Encoder-decoder	M3W, ALIGN	80B	Custom	Chinchilla
BLIP/2	2022/23	Encoder-decoder	COCO, Visual Genome	223M-400M	ViT-B/L/g	Pretrained from scratch
LLaVA-1.5	2023	Decoder-only	COCO	13B	ViT-L/14	Vicuna
CogVLM	2023	Encoder-decoder	LAION-2B, COYO-700M	18B	CLIP ViT-L/14	Vicuna
Gemini	2023	Decoder-only	Undisclosed	Undisclosed	Undisclosed	Undisclosed
GPT-4V	2023	Decoder-only	Undisclosed	Undisclosed	Undisclosed	Undisclosed
Qwen2-VL	2024	Decoder-only	Undisclosed	7B-14B	EVA-CLIP ViT-L	Qwen-2

Table 2.1: Architectural comparison of representative SOTA Vision-Language Models extracted from referenced paper, including some of the key models.

These are the models I have wanted to highlight, let's dive a little bit deeper into their interesting features:

1. **Flamingo**: represents a foundational step in the transition towards generative VLMs built on top of frozen LLMs. Its key contribution was the *Perceiver* architecture covered in section 2.1.2.1. This design, as mentioned, allows the pre-training knowledge to remain intact while the model learns visual context properties.

2. **BLIP-2**: introduced the *querying transformer* (Q-Former) as a bridging module between a frozen image encoder and a frozen LLM. The Q-Former designs often purposely structure tokens, manipulate query sparsity or introduce language or structure-guided query selection [16]. This training strategy involving first aligning vision and language representations and then connecting the Q-Former output to the LLM makes BLIP-2 one if not the most parameter-efficient models, as the majority of weights keeps frozen during training.

3. **LLaVa-1.5**: building upon the success of LLaVA, authors introduced the 1.5 version which incorporated several architectural and scaling techniques that enhance the performance. Compared to its predecessor, incorporated a broader and more diverse dataset and introduced more than 158k multimodal instruction-following pair examples for fine-tuning. Furthermore, it scaled images up to 336x336 pixel resolutions, enabling it to handle detailed visual content and capture finer aspects

of images [10]. It stands as a baseline for open-source VLMs by utilizing a simple linear projection layer as the bridging layer.

4. **CogVLM**: introduced the *Visual experts* architecture, able to reason complex spacial scenarios. Specially for robotic environments, it handles task of identifying relative position superb due to the layers in its transformers architecture destined for image processing.

5. **Gemini**: developed by Google, this model's defining feature is its massive context window, reaching up to 2 million tokens. This long-context capability enables the model to perform sophisticated temporal reasoning, identifying long-term patterns in agent behavior that models with smaller memory buffers would overlook. However, it's close source policy restricts the insight to its deep infrastructure.

6. **GPT-4V**: its strength lies in its world-grounding and common-sense reasoning, capable of interpreting high-level human instructions and translating them into logical sub-tasks. While its architecture remains undisclosed as the model mentioned beforehand, it serves as the primary benchmark for zero-shot performance in robotic task planning and complex scene description.

7. **Qwen2-VL**: latest version of the Qwen family, stands as one if not the most powerful open-source vision language models out there. Apart from supporting multilingualism, stands out understanding long duration videos and images of different resolutions, it has been the model I have been experimenting with in the practical area of the work, due to its versatility and lightweight sub variants which preserve the overall performance.

Looking at the models collected at table 2.1, there are some properties that show the evolution and shift from some tendencies to others. Among those trends, there are some that could be highlighted:

- **Connection paradigm shift**: there is a clear transition from alignment using *multi-layer perceptrons* (MLPs) to a deeper integration. In earlier models like *LLaVA-1.5*, the input images were treated as a set of tokens, whereas models like *CogVLM* introduce approaches such as the *Visual Experts* architecture. This architecture involves specialized visual encoders such as CLIP that provide distinct perception capabilities, enhancing the model's overall multimodal understanding.

- **Closed and open source gap**: proprietary code models like *GPT-4V* and *Gemini* lead the multimodal reasoning leader boards, however, their black box nature limits their reproducibility and fine-tuning. Even if we know they follow a decoder-only architecture, there is no available

information of the training data, number of parameters and their infrastructure. On the other hand, open source models offer a valuable insight of their properties such as the training data and number of parameters with total transparency.

- **Efficiency and scalability**: the most recent tendencies lead to *Mixture of Experts* (MoE) architectures. This technique allows to activate a portion of the total parameters during inference, achieving SOTA reasoning benchmarks while minimizing the computational cost.

2.8.2 Benchmarks

The amount of VLM benchmarks has grown rapidly since 2022, moving from basic visual question answering to include a wider range of tests that better evaluate the model's multimodal capacities from more aspects.

Category	Description
Visual text understanding	Model's ability o extract and understand texts within visual components.
Visual math reasoning	Tests the model's skills to sole math problems in image forms
Text-to-Image generation	Evaluates the image generation ability of the model
Hallucination	Evaluates the likelihood of models hallucinating on certain visual and textual inputs
Video understanding	Evaluates the VLM's ability to understand videos or sequence of images.
Visual reasoning, understanding, recognition and question answering	Evaluates the model's ability to recognize objects, answer questions and reason through both visual ant textual information

Table 2.2: Categorization of standard benchmarks used to evaluate SOTA VLMs.

Those are some of the 95 benchmarks covered in 13 categories for VLM evaluation collected in the mentioned study. Nevertheless, most of those VLM testing categories are still pretty far from the practical evaluation in the real world applications, such as scene understanding and autonomous driving.

2.8.2.1 Benchmark Data Collection

Those benchmarks are collected from a ton of datasets, the datasets used for vision language model benchmark evaluations are typically created using one of three data collection pipelines:

1. **Fully human-annotated datasets**: are created thanks to having humans collecting or generating adversarial test questions sourced from diverse subjects and fields. For some studies such as *Massive Multi-discipline Multimodal Understanding* (MMMU) [43], more than 50 college students belonging to different disciplines were gathered to extract questions from textbooks and reading material. The issue with this method of data extraction is the scalability and time consume problem.

2. **Partially human-annotated datasets scaled up with synthetic data generation and partially validated by humans**: Synthetic data usage has become handy to quickly scale up the dataset sizes. Workflows often consist on using human written examples to be fed to a LLM to generate more adversarial question and answer examples. In the case of VLMs, image and video data are often paired with captions, which are used by authors as context to prompt LLMs to extract answers and generate questions. However, LLMs are not always accurate and may produce unfaithful content or hallucinations without human supervision. To tackle this low-fidelity issues automatic filtering pipelines were developed and crowd worker validation was used.

3. **Interaction in the Simulator**: mainly targeted at VLM benchmarks in robotics, it gathers data for training and evaluation by assessing the VLM-powered agents online. Such a data generation method is applicable for the scenarios in which human-labeled datasets or synthetic ones are not feasible to acquire, and the data follows some common rules like physical laws. The outputs tend to the most robust, nevertheless, the potential *sim2real* gap might exist when transplanting the terminal VLM into real world-applications.

2.8.2.2 Evaluation Metrics

After collecting the data for benchmark evaluation, it's benchmark evaluation time. VLM benchmark evaluation metrics tend to be automatic to support repeated use at scale, and often influence the question formats used in the benchmark. The following are the most relevant metrics used for vision language benchmark evaluation.

- **Answer matching**: widely used for both open and close-ended question types. In order to evaluate the "perfect match" in practice, articles and unnecessary spaces are removed from both sentences

and check if the normalized predicted answer is contained in the normalized "gold answer". Natural language evaluation metrics such as *ROUGE,* F_1 and *BLEU* started being used, still their performance decayed when the generative LLMs started producing larger and more complex correct answers.

- **Multiple choice**: this metric involves selecting an answer from a set of possible correct options, options where often distractive answers are injected to confuse the model and enhance its reasoning capability towards these little tricks. Nevertheless, the evolution of the large language models have led to not necessarily needing to have the set of possible options in order to get the correct answer.

- **Image/Text similarity score**: score to evaluate the alignment across the generated image and their corresponding textual description, commonly used in the image generation benchmark. Using *CLIPScore* [12] for image-text alignment evaluation or *ROUGE* for caption matching. I want to stop on CLIPScore, being a cross-modal retrieval model trained on (image-text) pairs for evaluation purposes, the evaluation formula it uses is the following one:

$$CLIP - S(\mathbf{c}, \mathbf{v}) = w \cdot \max(\cos(\mathbf{v}, \mathbf{c}), 0)$$

where $\mathbf{c}$ and $\mathbf{v}$ are the text and image embeddings straight out from their respective encoders. Then the cosine similarity is applied to measure the distance between the two vectors and passed through an activation function to discard the negative similarity pairs and substitute them by value 0. Lastly, w is a scaling factor.

There is even a more advanced formula if dealing with references named *RefCLIPScore*. After passing the embedding of each reference through CLIP, the result is a set of reference embeddings R. Then, the score is computed by taking the *harmonic mean* of *CLIPScore* and taking the maximal reference cosine-similarity.

$$RefCLIP\text{-}S(\mathbf{c}, \mathbf{R}, \mathbf{v}) = \text{H-Mean}\left(CLIP\text{-}S(\mathbf{c}, \mathbf{v}), \max_{r \in R}(\max(\cos(c\mathbf{c}, \mathbf{r}), 0))\right)$$

Being the harmonic mean formula a kind of average used for ratios and rates:

$$H\text{-}Mean(x_1, x_2, \ldots, x_n) = \frac{n}{\sum_{i=1}^{n} \frac{1}{x_i}} = n\left(\sum_{i=1}^{n} x_i^{-1}\right)^{-1}$$

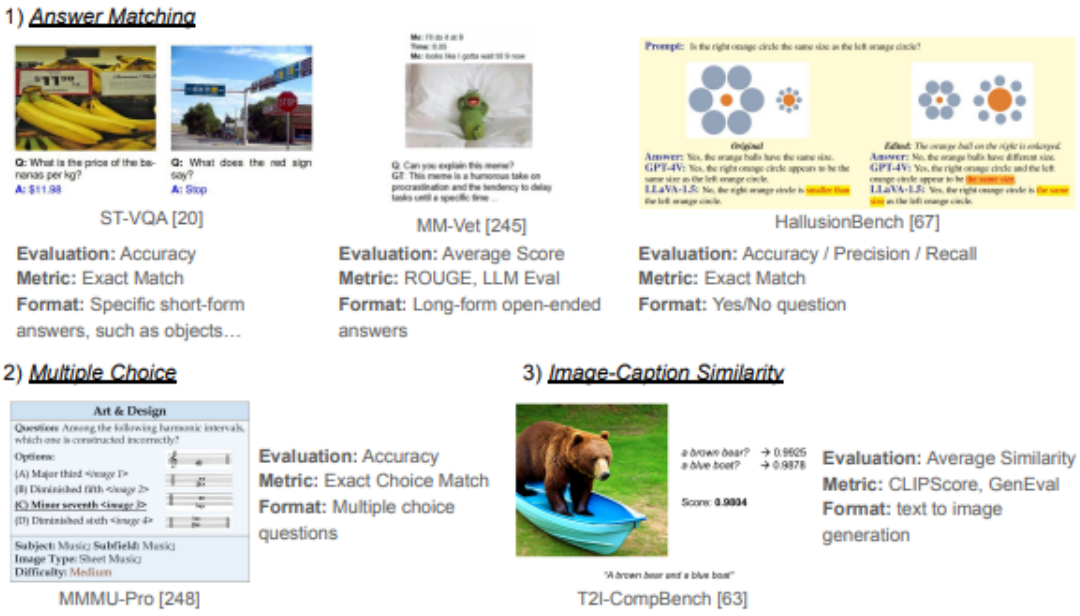

Figure 2.8: Most common benchmark evaluation metrics illustrated visually; showing *answer matching, multiple choice* and *image-caption similarity* graphical examples.

2.8.3 Vision Language Action (VLA) Models

As the latest step of VLM evolutions, a new sub variant arose with the name of vision-language-action (VLA) models. Multimodal foundational models which include all the prior work of the VLMs, adding the actions space to the vision and language ones. With these models, given an input image or video of the robot's surroundings alongside a text instruction, the VLA directly outputs low-level robot action instructions to be executed to accomplish the task [39].

The architecture of the VLA would stand as the standard VLM architecture just changing the decoder to an *action decoder* which translates the latent VLM representation into continuous output actions to be directly applied on the robot.

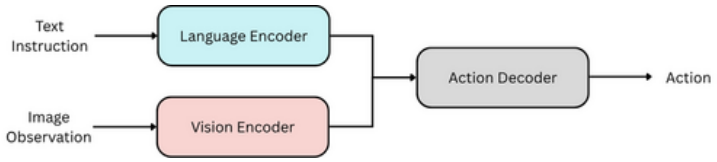

Figure 2.9: Standard vision language action model architecture.

2.8.3.1 OpenVLA

Among the most popular vision language action models, there is model *OpenVLA*, a 7B parameter VLA model trained on almost a million robot demonstration instances [15].

When it comes to the training of the model, a pre-trained VLM was used as a backbone and fine-tuned for robot action prediction. The input replicates the same (image-text) pair procedure seen in VLMs but the output now consists of a string containing the robot action to take. In order to get prediction by the VLM backbone, the robot actions are represented in the output space of the LLM by mapping continuous actions to discrete tokens used by the language model's tokenizer.

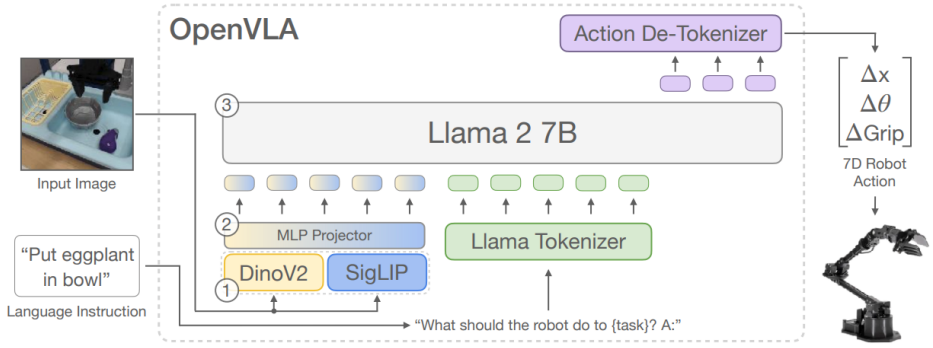

Figure 2.10: Given an image observation and a language instruction, the model predicts 7-dimensional robot control actions. The architecture consists of three key components: a vision encoder, a projector that maps visual features to the language embedding space, and the LLM backbone, in this case, a Llama 2 7B-parameter large language model.

However, current VLA models still face significant challenges regarding high-frequency control and multi-agent coordination. Most VLAs are trained for single-arm manipulation and struggle with the low-latency requirements of complex dynamic environments. In the context of this project, while these new vision language action architectures represent the SOTA in embodied AI, the integration of a vision language

models into multi-agent robotic systems represent a more robust combination, reason why I decided to stick to VLMs for the part of complex reasoning due to their stability over these newer models.

2.9 REINFORCEMENT LEARNING AND MULTI-AGENT SYSTEMS IN ROBOTICS

In the context of machine learning, RL is concerned with how an intelligent agent should take actions in a dynamic environment in order to maximize a reward signal. The agent must make decisions between trying new actions to learn more about the environment or using current knowledge of the environment to take the best action, balancing the exploration/exploitation trade.

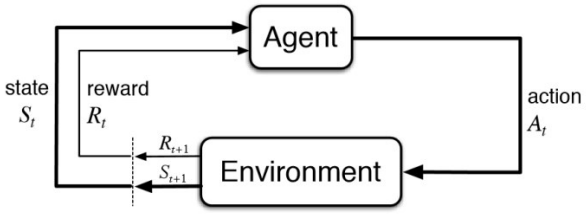

Figure 2.11: Basic RL worfklow schema: the agent iteratively maps states to actions by maximizing a cumulative reward signal through trial and error interaction.

Particularly their combination with deep neural networks, referred to as *deep reinforcement learning* (DRL) have shown remarkable capabilities in solving the most complex decision-making problems even with the high-dimensional observations in domains such as board games, video games, healthcare and recommendation systems [29].

2.9.1 Reinforcement Learning in robotic environments

Reinforcement learning offers to robotics a framework and set of tools for the design of sophisticated and hard-to-engineer behaviors. The successes of combining deep neural networks for reinforcement learning policy training underscore the potential of DRL for controlling robotic systems with high-dimensional state or observation space and highly nonlinear dynamics to perform challenging tasks that conventional decision-making, planning, and control approaches (e.g., classical control, optimal control, sampling-based planning) cannot handle effectively. Furthermore, robots need to complete tasks in the physical world, which presents additional challenges. It is often inefficient and/or unsafe for the RL agents to collect

trial-and-error samples directly in the physical world, and it is usually impossible to create an exact replica of the complex real world in simulation.

2.9.2 RL and VLM Combination

Reinforcement learning in complex scenarios is time consuming and as massive interaction steps are needed to learn optimal policies, and even if VLA seem promising to reduce time complexity, they still are immature and don't perform well on low-level control scenarios [41]. That is the reason why vision language models remain on the edge of helping models for reinforcement learning. Combining the reinforcement learning machine learning subfield with the architecture reviewed throughout the document, there has been an increasing trend towards combining both in order to form a more compact learning system. However, VLMs do not tend to control the agent's engine directly, rather they serve as an "orchestrator" which transfer their reasoning knowledge obtained from the environment to the agent. Vision language models can be applied into reinforcement learning as several approaches.

2.9.2.1 Vision Language Models as Action Advisors

In many studies vision languages models are proposed as action advisors for online reinforcement learning, leveraging the knowledge of VLMs to provide action suggestions for RL agents. Unlike previous methods, actions are proposed rather than designing heuristic rewards.

As illustrated in the comparison image below 2.9.2.1 for a robotic-arm grasp policy, zero-shot vision-language-action models tend to produce biased still inaccurate actions which partially approach task success. When it comes to the policy obtained in the early stages of a RL, often reflect near-random behaviors. In contrast to the previous two approaches, VLM as action advisor for online reinforcement learning (VARL) achieves the most accurate grasping with just a few interactions and converges efficiently.

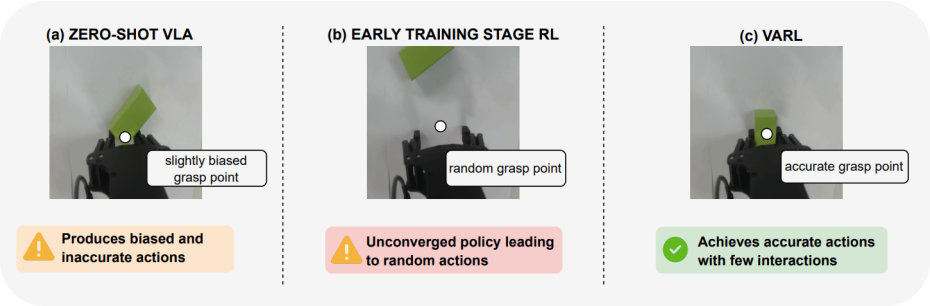

Figure 2.12: Different RL training approach comparison

2.9.2.2 VLM-based Reward Shaping

Another promising approach is to employ VLMs and LLMs as reward designers. Rather than suggesting direct actions, these models are used to shape or generate reward signals that guide reinforcement learning agents. Typically those rewards are generated by letting large language models read the task descriptions. *Eureka* and *Text2Reward* are some of the models which take that approach.

2.9.3 Multi Agent Robotic Systems (MARS)

Multi-agent robotic systems (MARS) are distributed autonomous systems where multiple robotic agents cooperate, compete, or interact to solve complex tasks that are too large, dynamic, or unpredictable for a single robot. Unlike centralized AI, MARS distributes intelligence across the whole network, allowing each agent to make decisions based on local information, which enhances scalability, adaptability, and fault tolerance.

Where the core challenge of MARS actually lies is in the coordination among agents, to address this modern multi-agent frameworks often go for one of two algorithm approaches: centralized and decentralized learning [42]. Centralized methods directly learn a single policy to produce the joint actions of all agents, whereas in decentralized learning each agent optimizes its rewards independently. *Centralized training and decentralized execution* (CTDE) algorithms fall in between these two approaches. This approach enables the agent to learn global information during the training phase in simulation while maintaining autonomy during execution. By using a centralized critic that observes the global state and the actions taken by the agents during training, the learning process becomes more stable. Nevertheless, the actors being the robots only have access to their local actions, enabling them to not rely on the critic once

deployed on inference.

2.9.3.1 Multi Agent Reinforcement Learning (MARL)

MARL addresses the sequential decision-making problem of multiple autonomous agents which operate in a common shared environment, each of which aims to optimize its own long-term reward by interacting with the environment and the other agents [44]. There are two different but closely related theoretical frameworks for MARL:

1. **Markov/Stochastic Games**: MDP generalization that captures the interconnection of multiple agents. In a Stochastic Game, the environment transitions between states based on the joint actions of all agents. Each agent receives a reward that depends not only on its own action and the current state but also on the actions of the others'. This model is particularly useful for representing continuous or long-term interactions in dynamic environments where the state is fully or partially observable at each discrete time step.

2. **Extensive-Form Games**: This framework represents the interaction as a tree structure, capturing the sequential nature of decision-making. Unlike Stochastic Games, Extensive-Form Games are ideal for modeling scenarios where agents move in a specific order or where there is imperfect information. For instance when some agents do not know the previous moves of others.

Multi-agent RL suffers from several theoretical challenges in addition to those raised in single-agent reinforcement learning. Being those the non-unique learning goals of the agents in the environment, the non-stationarity of the environment caused by multiple agents learning concurrently, scalability issues and structural information issues.

2.9.4 Sim to Real Frameworks

Sim2Real frameworks are computational systems designed to transfer control policies and models trained in simulated environments to real world applications. Generally implementing reinforcement learning in virtual environments, these frameworks seek model transferability of the knowledge obtained by the agents into the real world, trying to break that *reality gap* consisting of physical, visual and dynamic discrepancies between the simulator and the real world. Among the state of art frameworks, there are several advanced options to choose from.

- **Sim2Real-AD**: is a modular framework specially designed for autonomous driving that enables

zero-shot deployment of vision language model guided reinforcement learning policies [13]. The framework is known for implementing SOTA technology such as a *geometric observation bridge* (GOB) for converting monocular camera images into the simulator's *bird eye view* (BEV) representation and a *physics-aware action mapping* (PAM) to translate policy outputs into physical commands.

- **EAGERx**: provides a unified software pipeline for robot learning that supports multiple simulation engines and integrates state, action, and time-scale abstractions. The open source project available in GitHub [8], enables the user to define environments as graph nodes visualized in a GUI. Policies are trained in simulation and zero-shot evaluated on real systems and EAGERx's design allows users to create complex environments using modular compositions.

- **NVIDIA IsaacSim**: NVIDIA's physics simulation platform built on top of *NVIDIA Omniverse*, which will in fact be the selected framework for the practical field of the project. IsaacSim enables zero-shot sim2real transfer offering accurate physics simulation for realistic robot dynamics, synthetic data generation with domain randomization to train more robust models and it also supports reinforcement and imitation learning. On top of that it is integrated with *Isaac ROS* and *Isaac Lab* for real time deployment. Furthermore, Isaac Sim leverages NVIDIA RTX technology for ray-traced rendering, providing the photorealistic visual inputs essential for the effective reasoning of VLMs. The platform is built on the *Universal Scene Description* (USD) framework, which facilitates seamless asset interoperability and scene management. Crucially for multi-agent research, Isaac Sim enables massively parallel simulation by executing physics and perception pipelines entirely on the GPU, allowing for the simultaneous training of large robotic fleets in complex, shared environments. As a quick note, I have preferred to keep the overview of the framework simple in this section in order to deepen in the corresponding chapter due to its importance and links to the practical environment development.

In conclusion, this chapter has reviewed the transition from static vision-language reasoning to the embodied robot action. While current vision-language-action models seem promising, the complexity of multi-agent cooperation in critical environments requires a modular approach. The following chapters will detail how the reasoning capabilities of the selected VLM are integrated in multi-agent environments within NVIDIA IsaacSim to achieve coordinated robotic tasks.

3. SOLUTION APPROACH

3.1 SOLUTION PROPOSAL

In order to reflect the power of vision language models as high-level instruction orchestrators inside a robotic environment, I worked inside *NVIDIA IsaacSim*. An open source SOTA framework for robotics simulation and synthetic data generation in physically based virtual environments built on top of *NVIDIA Omniverse*. My final solution proposes a simulated robotic multi-agent environment where the vision language clearly dominates the high-level decision making, and orchestrates the whole workflow pipeline with the information received and sent to the other agents in the scenario.

3.2 SYSTEM ARCHITECTURE

Figure 3.1 illustrates the architecture of the project, divided into two main sides: the components that run on local and those running in the server. There are several reasons why I decided to not run everything locally, as running things remotely allows better VLM inference performance takins considerably much less time and also opens the gate for modularity if I decide to change the model or the server architecture.

3.2.1 Run locally

Everything that is run locally is located on the left side of the diagram. Basically consisting of all the physics, rendering and controller tasks.

3.2.2 Run remotely

While the things run in a server correspond to the nodes of the right side of the graph, encapsulating all the VLM inference processes. The VLM is run on the server because BentoML manages the VRAM resources more efficiently than if I were to run it locally.

3.2.3 Communication bridge

To connect both local and remote sections, I use HTTP protocols to communicate with a REST API. The local side of the architecture will send POST requests and the remote side will answer returning

JSON files containing everything needed to carry on with the workflow.

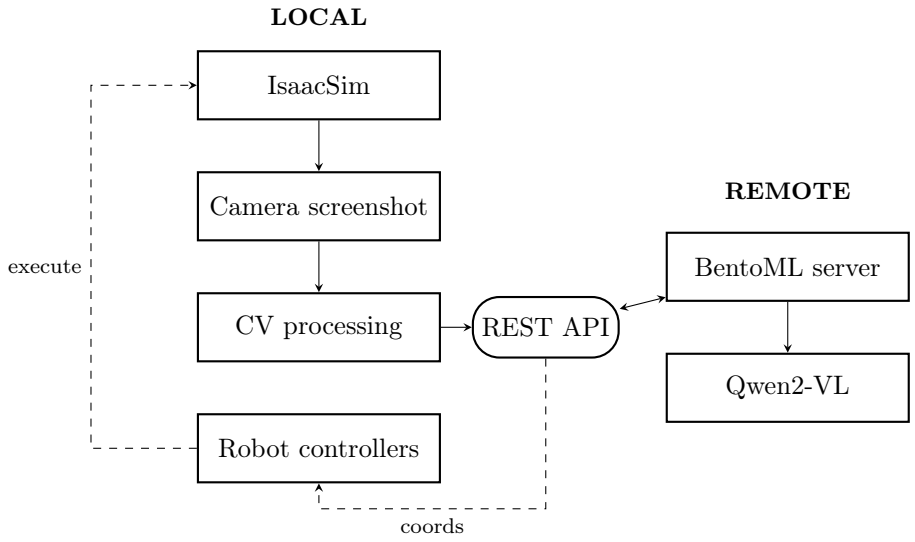

Figure 3.1: System architecture diagram.

3.3 WORKFLOW

The workflow used for the final scenarios inside *IsaacSim* could be reduced to this basic pipeline, illustrated below 3.2. The following workflow stands for the "standard" pipeline, however, depending on the desired task components of the pipeline will follow different procedures or repeat their selves more than one time.

1. **Visual captioning**: there will be cameras distributed throughout the scenario, pointing towards the key interest points where the VLM must put into action its reasoning capabilities. A screenshot of the area will be taken and sent to the next step.

2. **CV algorithms to enhance detection**: depending on the target object to be detected by the vision language model specified in the next component, a specific computer vision algorithm application script will be called. In order to modify the screenshot image so the specified target is more detectable.

3. **VLM hosted on BentoML**: there will be a *BentoML* server running up ready to capture the CV-modified screenshot of the desired section of the scenario taken in the previous step. However, as VLM receive a pair of (image-text) as input, the image will be accompanied by a prompt. Prompt consisting of identifying a specific target inside the scenario area captured by the camera. While

the service is running, another component will be responsible for coordinating the calls with the server's API.

4. **Target object coordinates prediction**: another script will interact with the VLM to try to get the closest coordinate predictions of the target object. Passing the prediction through hallucination filters and other several functions to get the most reliable function.

5. **Movement synchronization**: after getting the predicted coordinates, a dynamic movement script will be executed to get to those position. Depending on the robot agent and the task objective, the movement will end in itself or serve as bridge to another movement later on.

Depending on the task, there could be just one target object detection and movement cycle, or more than one as in the case of multi-agent scenarios.

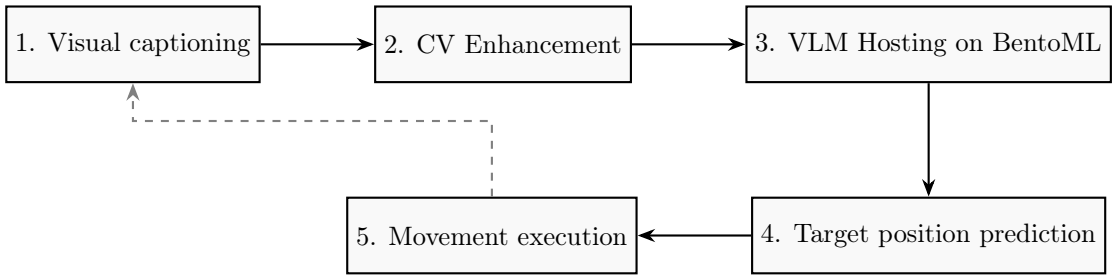

Figure 3.2: Standard workflow of the solution, going from the visual captioning stage up to the physical agent movement execution. The dashed line indicates a non-continuous feedback loop allowing for cycle repetition, especially in multi-agent tasks.

3.4 TOOLS USED

There are several technologies I have used in order to consolidate the final solution of the project. Nevertheless, some have evolved changing its versions or have been modified throughout the experimentation to include additional functionalities to seek better performance. Those tool's brief definition and implementation will be explained now, however, the different tweaks applied through the progression will be mentioned in the corresponding design experimentation section 4.

3.4.1 Vision Language Model choice

The VLM used in the final workflow was not the one I started experimenting with, there was actually a progressively evolution which led me from starting with the smallest models to scale up in the number

of parameters.

3.4.1.1 SmolVLM-256M - the initial pick

I actually started from the very smallest of the models, being the first model *SmolVLM-256M-Instruct*. Extracted from *HuggingFace*, the model is a tiny multimodal one, the smallest of the family. Originally designed for efficiency, its lightweight architecture makes it suitable for small applications just taking about 1.23GB of GPU RAM, reason why I decided to go for this model at first.

Nevertheless, its performance in the robotic simulator is what really pulled me back. While it would do great in simple classification or QA tasks such as the ones seen in section 4.1, it actually struggled in the task of accurately identifying the position of the target objects in the screenshot of the environment. Its performance when dealing with relative positioning of multiple objects in the environment was actually poor, so I decided to scale the power of the VLM just a little bit more, still not so much so it fit my hardware requirements. Being the next pick its big brother, *SmolVLM-500M* better performance and double parameters but still not good enough. After trying with these two models and their performance configurations, I decided to go with the final model: *Qwen2-VL-2B*.

3.4.1.2 Used Vision Language Model: Qwen2-VL

Already covered in the SOTA VLM models table 2.1, shows great performance as an open-source alternative. Furthermore, its different size alternatives suited perfect for my needs. As the biggest model sizes would not fit in my GPU's VRAM I have had to settle with the smallest one, being the *Qwen2-VL-2B-Instruct*. Some of the stages of the workflow pipeline were strictly run in the GPU and I had to find the balance of all the processes running simultaneously, meaning that I had to reduce the number of parameters of the model to its smallest in order to fit it all in the VRAM.

As shown in table 3.1 there are 3 size variants of the model itself, however, the number of parameters used in the vision encoder training remains constant and is just the LLM which scales up in number of parameters. This approach ensures the power of the ViT remains constant regardless of the LLM scale, meaning that there should be little to no more visual reasoning performance difference across the models.

Model Name	Vision Encoder	LLM Model	Description
Qwen2-VL-2B	675M	1.5B	The most efficient model, designed to run on-device. It delivers adequate performance for most scenarios with limited resources.
Qwen2-VL-7B	675M	7.6B	The performance-optimized model in terms of cost, significantly upgraded for text recognition and video understanding. It delivers significant performance across a broad range of visual tasks.
Qwen2-VL-72B	675M	72B	The most capable model, with further improvements in visual reasoning, instruction-following, decision-making, and agent capabilities. It delivers optimal performance on most complex tasks.

Table 3.1: Specifications and intended use cases for the different Qwen2-VL model scales, table extracted from original paper [32].

When it comes the architecture of the model and as that *2* indicates in the name, it was actually trained starting from the base point of its predecessor. In order to deal with scalability and performance issues they upgraded some of the components. They implemented a ViT able to deal with both image and video data formats and strengthened the LLM backbone. Several upgrades were implemented to strengthen the video analysis performance of the ViT.

- **Naive Dynamic Resolution**: this implementation enables the processing of images of any kind of resolution, dynamically converted into visual tokens. This was a result of removing the positional encoding layer from the vision transformer and introducing *2D-RoPE*, approach which replaces the traditional positional encoding by embedding absolute positions via dimension-wise rotations which allow naturally captured spatial relationships [26]. Technique used also in computer vision to capture the two-dimensional positional information of images. As statement, rotatory embeddings are made with rotation matrices and work by capturing both global and relative position information [27]. And rotation matrices are matrices are those which represent an angle rotation in the euclidean space.

- **Multimodal Rotary Position Embedding (M-RoPE)**: it uses a similar variant to method *2D-RoPE* mentioned in the previous paragraph in order to successfully model the positional information

of multimodal inputs. It works by decomposing the rotatory embeddings into new latent features such as height and width, so it can model explicitly the positional information of text, images and videos in LLMs.

Figure 3.3: Demonstration of M-RoPE

3.4.2 VLM hosted on BentoML server

Taking advantage of the material seen throughout the different subjects of the grade, I think it was suitable to host Qwen2 in a *BentoML* server and communicate with it via API calls. BentoML, is an open-source project implemented in a Python library for building online server systems mainly optimized for AI apps and model inference, being this second option where the VLM will really stand out. The inference will be controlled with REST API server lines, and will also be run in the GPU. As mentioned in previous sections, I have had to make my way through GPU space balancing, and did have to try out different parameters that make up the service to optimize performance, nevertheless all this configurations will be mentioned in the respective place. Among the reasons why I chose this option the following could be highlighted:

- **Computational efficiency**: IsaacSim requires significant GPU resources in order to maintain high-fidelity of the physics, and by delegating the inference part of the workflow to a BentoML service enables asynchronous communication and overall stability.

- **Resource management**: BentoML is known for a wise resource balancing, and as mentioned above, it was crucial during the development to manage my resources wisely. Hosting the model separately allowed deeper control on the VRAM allocation.

- **Availability and accessibility**: being open-source is often linked to having a great documentation and support from the community. Reason why I found no problems finding guidance and practical examples at their GitHub page [7].

3.4.2.1 Vision as a Service (VaaS)

This approach of deploying a vision language model in a cloud system has already been explored, it is called *Vision-as-a-Service* (VaaS). It was launched as a business model that leveraged the cloud system to apply SOTA computer vision tools to video streams captured in surveillance cameras [6], nevertheless VLM can be implemented equally.

Applied to the project, makes possible the interaction between the VLM and the robotic environment via HTTP requests and JSON responses and ensures scalability of the system for more complex environments.

3.4.3 IsaacSim as physics simulator

All there was to mention why I chose this framework was covered in section 2.9.4, so I will not repeat myself in this part. I just wanted to highlight this has been the robotic environment simulator I chose to develop the robotic scenarios.

3.4.4 IsaacLab for RL experimentation

IsaacLab is an open-source framework built on top of *IsaacSim* and *Omniverse* to inherit the high-fidelity physics simulation, aimed for unifying and simplifying robotic research workflows such as reinforcement learning, imitation learning and motion planning [17]. Also built to be run on the GPU, it is the ideal choice for sim-to-real transfer tasks in robotics. This framework is also modular and easy to customize, agile, open-sourced and includes various trial environments, sensors and tasks ready to be used from the very start. Furthermore, it has been the best choice for training RL policies inside IsaacSim. Offering the freedom to create custom training environments or train tasks that are already included. I must state I have used the `main` branch version of the repository according to the IsaacSim version I have been using.

The reason why I chose this framework among other options such as *OpenAI Gym* or *PyBullet* is not just because of its direct integration inside IsaacSim, but also because of the following points:

- **Parallelization**: the fact of IsaacLab running on GPU allows optimized training techniques such as parallel training several environments simultaneously. Moreover, this frameworks also allows running inference in the GPU, discarding any CPU bottleneck issues I have.

- **Available examples**: not all robots have training examples available at the repository, but Franka does. Fact which reduced what would cost a lot of time designing observation and action spaces

and specific rewards.

- **Modular design via managers**: the framework is designed as a manager-based API which organizes environments into reusable and composable components, enabling consistent workflows across diverse tasks. There are manager classes for the scene, observation, reward and termination components of the environment, allowing modular swaps, changes and class inheritance.

- **Integration of USD with PhysX in OmniPhysics**: This mixture provides a representation of the objects and robots in a hierarchical manner in the scene. The scene, represented in a graph is then parsed into *PhysX* allocating all the internal representation of the environment as tensors to the GPU. PhysX being a physics engine developed by NVIDIA originally used for video games. Once the representation is in a tensor manner inside the GPU, IsaacLab interacts with that info through View APIs, allowing reading or writing objects of the scene while managing data efficiently and improving usability.

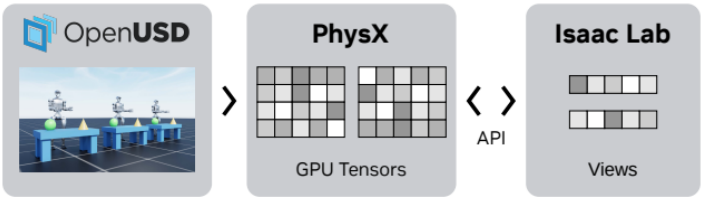

Figure 3.4: Procedure of how IsaacLab reads the environment. First converting to tensors the hierarchical representation of the environment, sending them to the GPU and manipulating those tensors via Views API calls.

Finally, I will be using *stable-baselines3* (SB3) framework for all the trainings due to its support inside IsaacLab and compatibility with visualization tools such as *Tensorboard*.

4. DESIGN AND IMPLEMENTATION

4.1 INITIAL DUMMY EXPERIMENTATION

Before designing what would be the final orchestration scenario where everything would be synchronized and coordinated, I wanted to start from the very basics, the modular components. Before integrating the vision language model into the simulator framework, I preferred to experiment with the code available in the documentation. In order to gain an insight on how a VLM works from scratch, I did several experimentation *notebooks* to practice with the basics.

4.1.1 CLIP Demo Analysis

The first modular practice I performed was experimenting with the *CLIP* library and code available at the official GitHub repository [21]. I made a simple classification task notebook where the contrastive properties of CLIP would really shine. Consisting of a classification of an image among a set of custom labels and secondly a *zero-shot classification* to check the zero-shot capabilities of the model. All of this code is available at path `/Scripts/Demos/` inside the project directory.

4.1.1.1 Simple Classification

Using the *ViT-B/32* model CLIP offers inside its library, being a vision transformer for image classification, I have wanted the model to classify an image among a set of custom labels that image may belong to. As referenced in image 4.1.1.1, the image to classify is a portray of a surfer riding a wave. The model must use its contrastive learning techniques it was trained with, and distinguish the image from a possible set of candidates being: a surfer (correct answer), a truck (incorrect) and a mountain (incorrect).

Figure 4.1: Image to classify used in the CLIP features demonstration notebook, showing a surfer.

After processing and encoding the features of the image and *tokenizing* the labels of the candidates, calculates the probabilities of the image belonging to each label. Returning a probability distribution that sums up to 1 because of the *softmax* activation function. Looking at the results, the vision transformer trained with CLIP's contrastive learning approach had no problem identifying the winner. Reflecting CLIP's astonishing performance for simple classification tasks, however, the second sub-task is a little bit more difficult.

Label probabilities: [[9.9999571e-01 2.2835159e-06 2.0020473e-06]]

4.1.1.2 Zero-Shot Classification with CLIP

Loading the *CIFAR100* well-known dataset for including thousands of colored images belonging to 100 different classes, the same vision-transformer must generate text descriptions for all of the 100 classes of the dataset and taking a random sample instance calculate the similarity between the image's visual inputs and each of the generated descriptions. Basically a zero-shot classification task, where the model must predict the answer without any previous knowledge, just based on the generated text captions.

Selected image: `snake image`
Top 5 predictions:
`snake: 65.31%`
`turtle: 12.29%`
`sweet_pepper: 3.83%`
`lizard: 1.88%`
`crocodile: 1.75%`

Great results as the model gets the correct answer sharing a pretty well distributed probability. As while getting the best result the next most probable ones are all green animals or green food, showing great zero-shot classification performance.

4.1.1.3 Image Captioning with BLIP

I also tried other tasks as image captioning with other VLM models such as BLIP. Having the same surfer image 4.1.1.1 as input, the model did describe it surprisingly well for a maximum number of tokens like 30. Being its short description for the image:

Caption: *a man riding a wave on top of a surfboard*

4.1.1.4 Visual Question Answering

Question answering also returned great results. Still using BLIP and the same surfer photo, I asked the model a rather complicated question.

> **Question**: *What color is the surfer's board?* **Answer**: *white*

The question of what is the color of the surfboard seems easier than what it actually is, as the model must be able to distinguish between the board and the white foam parts of the waves. Which is already struggling to the human eye. Even if the question was tricky reason why I chose both the question and this image, the model got it correct once again.

4.2 VLM SCENE CAPTIONING IN ISAACSIM

Even if the project's goal is to implement an end to end VLM integration pipeline into a robotic environment, IsaacSim already offers some kind of vision language model integration inside the framework [19]. In this section, I will go over the capabilities of this functionality and argue why I still chose to develop my own pipeline.

The VLM integration the framework offers includes detailed descriptions including object relationships and spatial reasoning of an environment returned in a graph format. Being able to translate complex three dimensional spatial relationships into a compact graph while maintaining all the relevant information. In order to download and start using this add-on, I recommend installing it in the extensions panel by the name `Isaacsim.Replicator.CaptionCore`.

4.2.1 Workflow of scene captioning

Similar to my workflow developed, this process is also pretty straightforward and lineal. The workflow flows as follows:

1. Environment initialization.
2. Scene capturing by a virtual camera inside the environment.
3. VLM processing. The image is fed to a VLM to understand the object relations.

4. The model generates a brief phrase describing the environment. Being the result a description string.

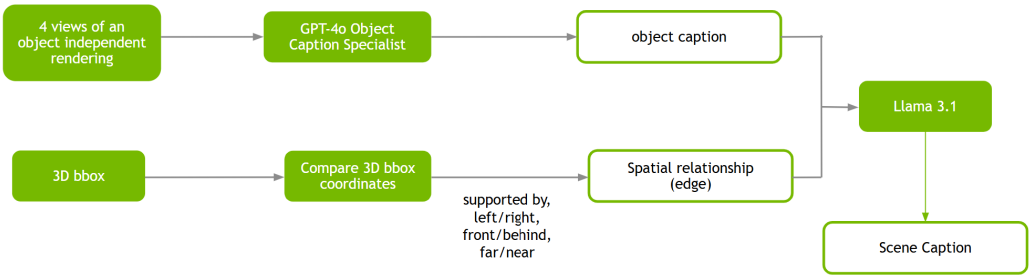

Figure 4.2: Workflow of the VLM scene captioning add on available at the IsaacSim documentation.

4.2.2 Scene representation using graphs

As mentioned earlier, all the information of the scene is captured in a single graph, which is returned after each running iteration. In the graph, the nodes represent the objects and the edges stand for the spatial relationships among them. Besides that, the graphs also capture data such as relative positions and even orientations, making them extremely useful for scene capturing returning an organized graph. Let's put it into practice, for this demonstration I will load an environment which came pre-installed alongside the framework.

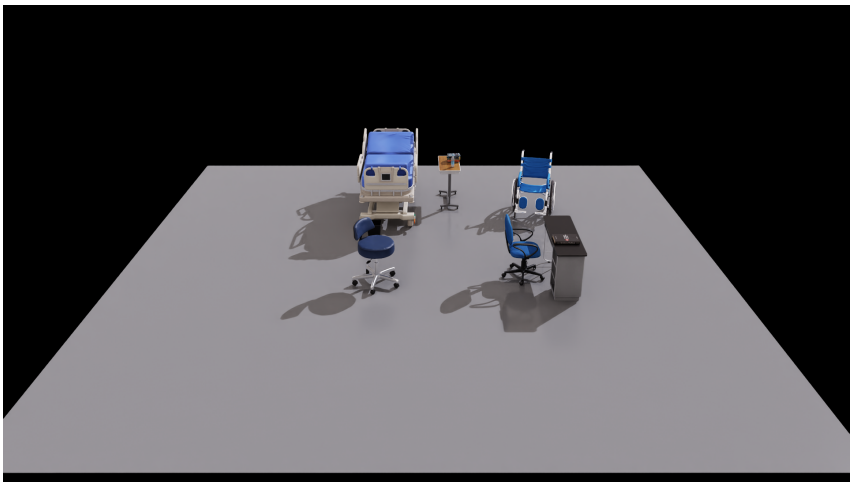

Figure 4.3: Environment used for the VLM scene capturing. Consisting of a hospital room containing objects we could find in an hospital. Such as the patient's bed and the doctor's table.

After applying the scene captioning technique to the environment seen in figure 4.2.2, we should get a number corresponding to each object in the picture representing a node in the graph. And links representing the graph edges which stand for the spatial relations among them.

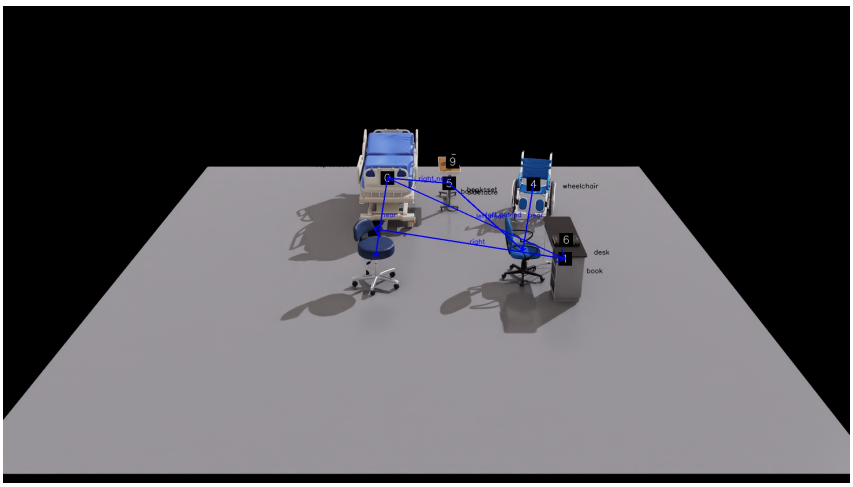

Figure 4.4: VLM scene captioning result applied to the hospital room scene, with numbers representing object nodes and lines standing for the graph edges.

4.2.2.1 Returned results

All of the information captured in the scene graph is returned in three different JSONs, alongside the 4.2.2 image.

1. **full_scene_graph**: this JSON includes the complete information captured by the VLM. Includes the 2D and 3D automatically extracted bounding boxes of the objects plus identifiers and other metadata.

2. **pruned_scene_graph**: the second file prunes the irrelevant relations captured and present in the first JSON. Simplifying the content and maintaining the essential things. While this one still maintains the bounding box data is a greater option for analysis as it presents a much cleaner structure rather than the pure chaos the first JSON has.

3. **scene_graph_caption**: for each object identified in the scene, it returns a caption with a detailed description of the object and a list of the objects it is related to. These related objects tend to be the closest in the scene. For instance this is the example for the hospital bed:

```
"0-hospitalbed": {
  "class": "hospitalbed",
  "caption": "A hospital bed with white frame, blue mattress, and
      multicolored control buttons. The bed has wheels for mobility
      and various adjustment mechanisms.",
  "id": 0,
  "relations": [
  "near 2-chair",
  "to the near left of 1-desk"
  ]
},
```

Pseudocode 4.1: Scene graph caption result for the patient bed in the hospital scene.

As we can see, the VLM made an outstanding description detailing the material colors and the structure of the bed. And the related objects stand for the link lines seen in figure 4.2.2 being the chair and the desk.

4.2.3 Comparison with my developed workflow

Although both workflows share similarities in their procedure and results returned, there are several differences which led me to develop my own system. The following are the main reasons that helped me decide on the best alternative for the project:

- **Extraction results**: while NVIDIA's scene captioning returns all the information of all the objects included in the environment, has no additional idea of what to do with their descriptions and relations. On the other hand, my pipeline includes the additional task which helps the model know what to do with the extraction data. That additional information is provided by the target prompt i.e. "locate the red cube...". Additionally my model must filter all the distractive noise of the environment to focus on the target, while IsaacSim's captures it all, and I think that when working with models with limited capacity is a smarter choice to filter the unnecessary to optimize performance.

- **Modular and independent architecture**: IsaacSim's scene captioning plugin is directly integrated into the simulator, and it cannot be modified. Whereas my architecture establishes the VLM in a server, making it more portable, lighter and more independent. Furthermore, I am able

to tweak any parameter of the model or even change it, while NVIDIA's comes pre-established and is not open for any changes. In these situations I always prefer to go for the modularity choice. Hosting my own model also introduces the opportunity of enhancing its performance as I have with computer vision, however that is not an option for the replicator choice.

- **Model choice**: nevertheless, not everything have been advantages. The models used in the official replicator pipeline 4.2.1 are much more powerful than the one I have used, leading to greater performance. The model used by NVIDIA is actually *GPT-4o*, and even if the number of parameters has not been officially published it could be around 200 billion parameters, almost 100 times more than the open-source alternatives I have used, and in addition, *Llama 3.1* is another possible alternative which can scale up to 80B. It is evident the previous models will considerably outperform my alternative, nevertheless, I am not able to run that computation power to host the models locally. Reason why I went for a more humble open-source model with much less parameters. Which, even if its performance does not catch up, with some configuration can get considerable results as it has done. Moreover, I think it was a better choice to go for the pure VLM model and discard any "black box" alternatives.

4.3 TARGET DETECTION TASK

The target detection went through progressively changing and modifying parts of the BentoML server script to achieve better performance. Reason why it is vital to mention the structure of the service, this is what the server is made of and its functionality progressions over time.

4.3.1 BentoML service class - VLMServiceIsaac

Responsible for loading the VLM pre-trained instance and specific processor, running in the GPU, I decided to deal with *torch.float16 dtype* to balance performance and stability. I also experimented with sending the API calls to the CPU and setting the *dtype* to *torch.float32*, but due to the optimization of Bento in the GPU, the result were always time-outs even if float32 promised better results.

This class also has some functions for filtering the VLM hallucinations, as the small size model often fails and returns disparate values. Depending on the desired target type's size, it will consider some dimension limits to discard hallucinations. This hallucination filters takes the *bounding box* of the object and compare its viability, that bounding box consists of a rectangular region which embodies the object. Consisting of coordinate points y_{min}, y_{max}, x_{min} and x_{max}.

Then there is the main function *infer* which as as the bridging between the robotic environment and the VLM. The function receives the environment cropped image, the instruction it must follow and the target type, and returns the coordinates of the desired target if found. It works with the usual template VLMs receive, being the same image-text pair discussed throughout the whole documentation see pseudocode 4.2.

```
"role": "user",
"content": [
{"type": "image", "image": image},
{"type": "text", "text": prompt}
]
```

Pseudocode 4.2: Structure template Qwen receives, consisting of the screenshot of the desired part of the environment and prompt. Similar to a JSON format.

The prompt the service receives has been changes several times, each time to the overall target detection logic. This has been the end result:

```
prompt = (
f"Locate the {instruction}. I have marked its exact color center with a
    yellow square. "
f"Focus ONLY on the center of that yellow square to provide the
    coordinates. "
f"Ignore all black shadows or robot parts. "
f"Return only JSON: {{\"target\": {{\"bbox_xyxy\": [ymin, xmin, ymax, xmax
    ]}}}} "
f"using normalized coordinates 0-1000."
)
```

Pseudocode 4.3: Custom prompt the VLM receives, indicating what to detect, what to return and the format of the returned data

The instruction is the target object to detect in the image, and the yellow square helping technique is highly related to the computer vision section 4.3.4 next seen in the documentation. The other lines of the prompt consist of helping lines to ensure the VLM focuses on the desired target and the format it must return the target coordinates. Normalized *bounding boxes* to be filtered with the previous hallucination method. The next lines of the function are dedicated to the call preparation and output parsing to clean

the noise.

4.3.2 Evolution of detection functions

The second part of the script consists of the detection function that will be called in order to perform the target detection, and it's the part where there have been the most changes depending on the approach I have followed.

- **ground**: function used in the first server scripts for the very basic environments. The main difference between this function and its successor is that this simpler approach just detects a single target in the given image, while the following detects all of the possible objects and then filters to get the target. I used this function for the environments from the beginning where there was not so much difficulty, but I decided to change it for the more complex environments where it showed a catastrophic performance when dealing with more objects because it always returned hallucinations or mismatched the target object.

- **multi_ground**: this second and definitive detection function uses more sophisticated techniques and was used for the final multi-agents scenarios, showing much better performance than the previous. Depending of the target type, it uses additional helping prompts and uses them to test different coordinate predictions to then get the median of those coordinates as final prediction. I chose the median instead of the average to deal better with those hallucination outliers. Those custom prompts help changing the 'subject' of the prompt (basically being the target described with different names) so the model can have a broader idea of what he might be asked to detect. Those custom prompts will be found in the appendix 7.3.

The BentoML created service will be run and interacted with inside the robotic environment simulator for the target detection tasks. In order to mount the service just need to run the following command 4.4:

```
1  bentoml serve server_bento:VLMServiceIsaac --port 8000 --reload
```

Pseudocode 4.4: Command to start the created VLM service

4.3.3 Task definition

The target detection task, reduced to the basic, consists of calling the VLM service with API calls selecting the target to be the detected and wait for its response to get the target coordinates approximation in a JSON format. I decided to store the coordinates in a file for re-usability and modularity for other scripts/tasks. Although it is the VLM the one getting the coordinates predictions, it is run inside the IsaacSim simulator, so I had to adapt the capturing logic to the camera properties of the framework. Among the properties I had to take into account:

- **Display resolution**: display dimensions of the screen I work with, necessary to calculate projection and unprojection functions, `(1280, 720)`.
- **Focal length**: `(18.14764 * 1280) / 20.955` hard-coded camera property which servers for defining the camera zoom movement. The higher it is the narrower view the camera will have, while the lower the wider [18].

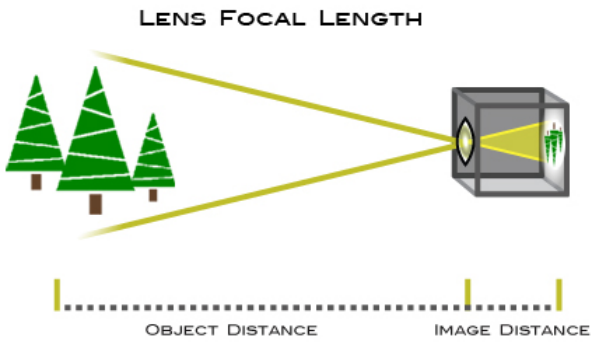

Figure 4.5: Focal length of the camera, relation between the object distance and image distance.

- **Principal point** (C_x, C_y): `CX, CY = 640, 360`. Refer to the optical center of the image, acting as a (0,0) origin point.

4.3.3.1 Procedure

1. Capture the scene screenshot in the given resolution and RGB channels and camera metadata. Data such as the relative distance between each pixel to the camera inside the environment, these distances then must be transformed to real ones to don't mismatch units.

2. Then the image is sent to the VLM opened service, and choose the desired corresponding detection function (ground/multi_ground) endpoint URL. Next receive the bounding box response.

3. The received coordinates do not share the same dimension to the real scenario, so it is essential to convert them from 2D to 3D coordinates, often called *unprojection*. I have employed two formulas to deal with the transformation of coordinate dimensionality. The first formula converts pixels at 2D position (u, v) to its 3D position relative to the camera:

$$x_c = \frac{(u - C_x) \cdot Z_{depth}}{f}$$
$$y_c = \frac{(v - C_y) \cdot Z_{depth}}{f}$$
$$z_c = -Z_{depth}$$

And the second takes the 3D coordinates of the object relative to the camera and maps it to the position inside the scenario inside the simulator.

$$\mathbf{P}_{world} = \mathbf{T}_{cam} \cdot \mathbf{P}_{local}$$

Where the variables used in both formulas stand for:

- (u, v) represent the 2D pixel coordinates in the image plane.
- (C_x, C_y) is the principal point, which indicates the optical center of the image in pixels.
- Z_{depth} is the perpendicular distance from the camera plane to the object, extracted from the depth map.
- f is the focal length of the camera expressed in pixels, variable explained previously.
- $\mathbf{P}_{local} = [x_c, y_c, z_c, 1]^T$ is the position vector in homogeneous coordinates relative to the camera's reference system.
- $\mathbf{T}_{cam}$ is the extrinsic transformation matrix of the camera (a 4×4 matrix encoding the rotation and translation within the simulator). Extrinsic matrices are those who position the camera

in the real world/scenario, while the intrinsic are the ones used for focal and principal points [25].

- $\mathbf{P}_{world}$ is the resulting vector containing the final (X, Y, Z) coordinates in the IsaacSim global 3D coordinate system.

4. Depending on the target being larger/smaller, I have set a stability counter to make sure the final result is a consolidation and not a random disparate value. For smaller targets such as the cubes, I have set more strict thresholds to ensure getting right the coordinates of such tiny targets while for larger objects like the platforms I have been more flexible.

5. Save the desired metadata in JSON format. Consisting of information about the target type, the color, the position in the scenario, the relative position to the other targets and the camera used.

While this approach seemed correct and feasible, the results were a little bit disappointing. Perhaps due to the VLM's lack of power, the predicted coordinates visualized in the environment with a custom marker spawning function, usually were too far from convincing or were simply disparate. This was were I thought of implementing helping techniques to boost detection performance.

4.3.4 Computer Vision for performance enhancement

CV algorithms were the first idea that came up to me, so I decided to make a separate detection script for each target type. In the procedure, I had to deal with the other components that were part of the scenario such as the shadows of other objects and the lightning produced by the configured light source. These were the steps followed for each target case and available at its respective file:

- **Cubes**: due to the tiny size of the cubes I defined a range of threshold for each color to be distinguishable from the other components of the scenario. I also converted the image to *HSV* for better segmentation performance. In the case of finding the target(s), I defined the contour yellow rectangle surrounding each target, same yellow rectangle specified at the prompt the VLM receives. The initial screenshot of the cubes analysis can be found in the following image.

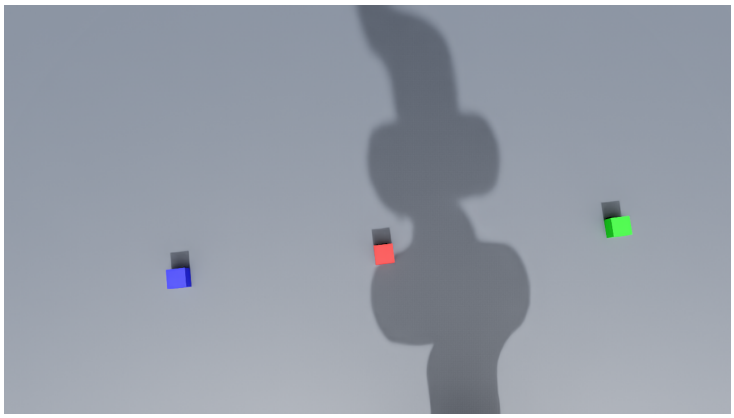

Figure 4.6: Raw image sent to VLM for cube analysis

After the initial CV filter application, the result is the following:

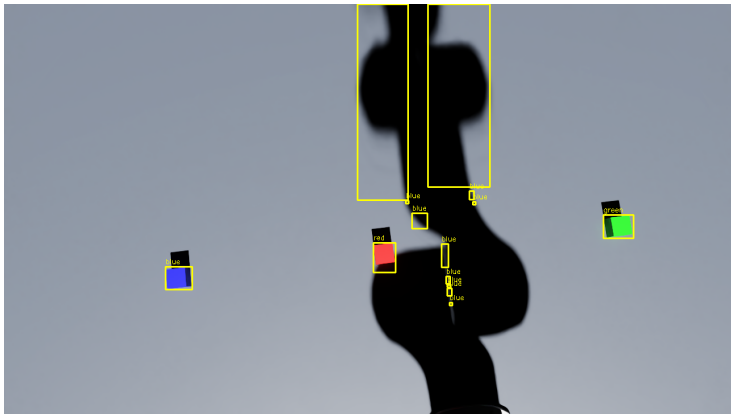

Figure 4.7: First CV applied iteration

As seen in in figure 4.3.4, there are so many things that can distract the VLM as this is the image itself sent to the server in the image-text prompt. So I needed to apply further cleaning to get a simpler result without confusing text and shadows.

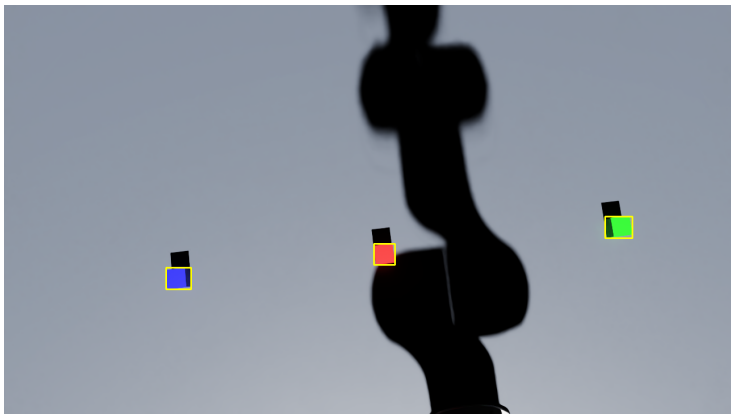

Figure 4.8: Final CV application iteration

This approach returns a much cleaner analysis where there are no distractive shadows nor cube names, just the image with the yellow rectangle surroundings.

- **Palettes**: the platforms were much simpler to detect due to their much bigger size, nevertheless, there is an inconvenient the VLM must resolve which did not have to for the cubes case: there is an "obstacle" that must avoid detecting. The obstacle being a robotic arm between the platforms.

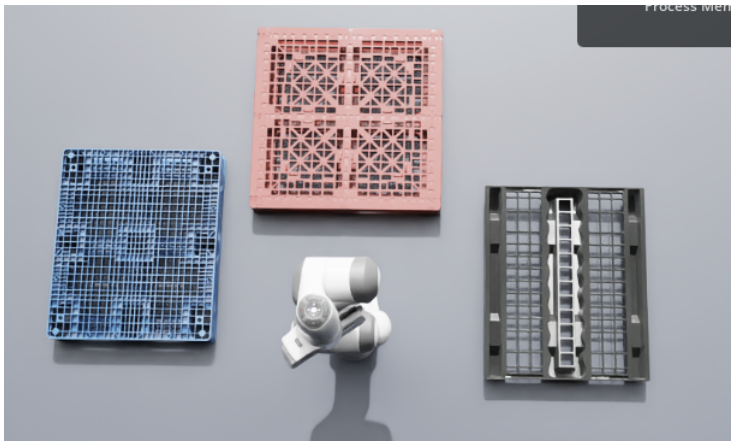

Figure 4.9: Raw image of palette detection

The script uses the same color range technic as before but now transforms the image to *HLS* to filter the white part of the distracting obstacle. In order to avoid detecting the robotic arm instead of the palettes, I create a mask to cover the undesired part of the image.

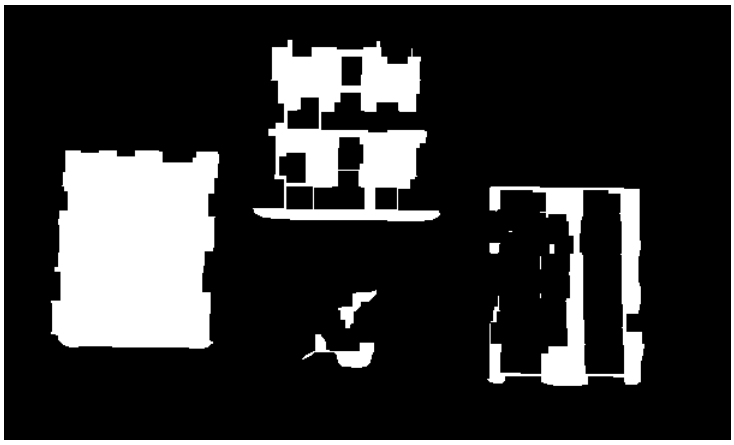

Figure 4.10: Result of masking the palettes

As seen in image 4.3.4, the mask does not behave equally for all of the platforms, as each of the palettes has a distinct hole distribution the result is different for each. While the blue one shows the most compact surface with the smallest holes obtaining the best overall masking result, the remaining show not so complete masks as they originally have bigger holes and more complex designs. The final result after applying all of the filters is the following 4.3.4:

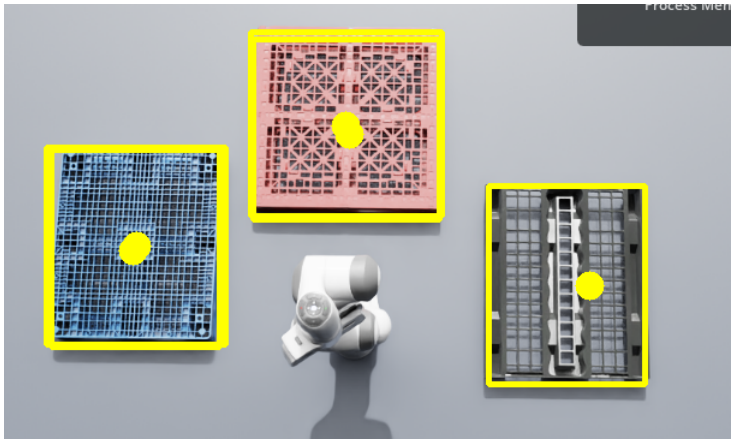

Figure 4.11: Palette detection results with CV application

The key here is to try to get the center of the platform, as the objects to place on top of them are so little they may fall from the holes. The final coordinate predictions of the target will be those yellow circles near the centroid.

- **Robotic agents**: finally this is the task where CV struggled more to shine. In image 4.3.4, there are several agents and environment components while its the robots what the VLM must target.

Figure 4.12: Raw image of robotic arm detection

The same way as in the previous case, I apply masking to "get rid of" the unnecessary parts of the environment, removing the majority of the middle section of the image and applies the centroid circle for each robot.

Figure 4.13: Mask filtering for the robotic arms image

The masking gets rid of the other agents which stands in the middle of the arms, and successfully removes the cubes and platforms. It captures the overall structure of the arms, as it was too difficult to get the complex curves and shapes of the whole robot. Still, the results are great, let's check the final result of the robotic arms target detection task alongside CV filters.

Figure 4.14: Final result of the CV algorithm application to the last target detection case.

4.3.5 Helping offsets

While CV filter application significantly boosted the difference between ground truth target position and the VLM predictions, there was still some tendency of deviating a little on the prediction for some of the targets. While smaller VLM such as the one selected perform great for segmentation and target detection, they still struggle when returning predicted precise position coordinates. Furthermore, when dealing with small objects, the error margins may lead to imprecise predictions, reason why I decided to implement the last helping techniques of the target detection section: calibration guiding offsets. Taking as reference predictions visualized by the spawned colored-markers, I introduced a set of guiding (x, y, z) offsets to guide the VLM through the predictions it struggled most with. The application of the offsets basically consists of a sum of the prediction array and the offset one.

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix}_{final} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}_{prediction} + \begin{bmatrix} offset_x \\ offset_y \\ offset_z \end{bmatrix}$$

This simple trick enhanced the performance of the target detection alongside the computer vision algorithms to the definitive version, reaching the best performance and most accurate detections. The offsets were found by comparing the predictions to the ground truth several times and taking the ones which generalized better, they are available a the appendix at section 7.1.

4.4 AGENT CONTROL

The robotic agents used throughout the development needed to have movement functionalities. Controllers with which they could use their joints to move somewhere, close their grips or do any other

physical actions. In my case the robots used were the *Franka arm* and the *Jetbot*, one used for lifting and object manipulation purposes while the second being primarily optimized for navigation tasks (will be explained deeper in the corresponding section). For the first robot I used a pre-defined controller which worked flawlessly, but for the *Jetbot* I had to manually craft one controller class as I did not find any convincing ones. Both controllers follow the same principle, having a primary function which receives (x, y, z) coordinates and somehow travels there. In this section of the documentation I will go over the steps and progress I made to successfully develop the control managers of both robots.

4.4.1 RMPFlowController: the Franka Controller

RMPFlowController implements the *BaseController* interface from the motion generation API library in python [20]. And it is the best controller choice for reaching specific targets while automatically adjusting the position and orientation of their joints and avoiding obstacles. It uses a pre-defined interface to access obstacle information for avoiding collisions. Furthermore, this interface will avoid Franka to crash its hand against the other component of the robotic arm, automatically avoiding collisions and finding the best way through.

After importing the necessary libraries to form the controller class, I developed the essential functions that every controller must dispose of.

- **move_to_cube_top**: function which receives the target (x, y, z) position and brings its "end-effector" (its "hand") there with the desired top orientation and keeping the grippers closed. In order to stay just on the top, I configured a safe distance which can be arbitrarily modified so it does not crash with the object found at the given coordinates.

- **open/close_gripper**: luckily, this controller library offers a binary function to open/close the end effector's grippers. I will call the respective function when needed.

4.4.2 Custom Jetbot controller class

Whereas the simple import of the predefined controller initialized all the existing joints of the Franka arm, I needed to manually set the joints to their specific locations. Being the wheels and chassis. There is a function that also receives the target coordinates and travels there, however, the movement logic follows the principle of first rotating to the correct angle facing the target and then moving forward. It first calculates the difference between the error between the robot's current position and the goal coordinates

(gx, gy), and depending on the error it spins the left or right wheel to face the goal target direction in a straight line. This is where I developed the following functions:

- **quat_wxyz_to_yaw**: IsaacSim uses *quaternions* (w, x, y, z) to represent orientations, but are hard to represent and use for programming tasks. This functions converts quaternions to angles in radians $[-\pi, \pi]$. I use the term *yaw* to represent the rotation around the vertical Z axis).

$$((\theta + \pi) \pmod{2\pi}) - \pi$$

  where θ is the angle received by the function.

- **wrap_to_pi**: used for calculating the orientation error between current and target vectors and update the corresponding wheel velocity to face the target.

Once orienting correctly and facing the target, it is time to go forward. For this, each time step the euclidean distance is calculated and a proportional speed is adapted. Meaning that the further it is from the target the faster it will go, while the closer it is to reaching the destination the slower it will go, gradually decreasing the velocity. This functionality will come handy as in some tasks the *Jetbot* must carry objects on top of it, and a high velocity will lead to the objects falling. Furthermore it will stop once reached a safe distance to the destination, as depending on the task it will have to stop far enough so Frankas can manipulate the objects the robot carries.

4.5 ISAACSIM ENVIRONMENT DESIGN

Once collected all the necessary components, it was time to put them all together inside a controlled simulation environment. As I had no previous experience with the framework, being *Unity* the most similar experience I have had, I decided to progressively develop more and more sophisticated environments to try out my VLM integration pipeline, starting from the very simplest. The following subsections will relate my progression and evolution while adapting to the challenging learning curve IsaacSim has, detailing how I made my first scenarios, what they were made of and what techniques I applied in each.

4.5.1 Components of the scenarios

Before I start, I think it is a better idea to first list the components the scenarios have made with and talk about their properties, and then just say what each scenario is composed of.

4.5.1.1 Franka Panda - Robotic Arm

The robotic arm which can be found at inside the `Assets/Robots` path in the asset explorer tab inside the editor, is the digital representation of the real Franka Emika Panda robot arm [24]. Originally it is composed of 7 axes and is used for manipulation tasks, being able to grasp objects up to 3.0kg and the robot itself weighs around 18kg. In addition to the original seven joints, IsaacSim adds the end-effector to the robot, basically being the "fingers" of the arm see figure 4.5.1.1. The names of the joints are the following: *panda_finger_joint1, panda_finger_joint2, panda_joint1, panda_joint2, panda_joint3, panda_joint4, panda_joint5, panda_joint6* and finally *panda_joint7.´*

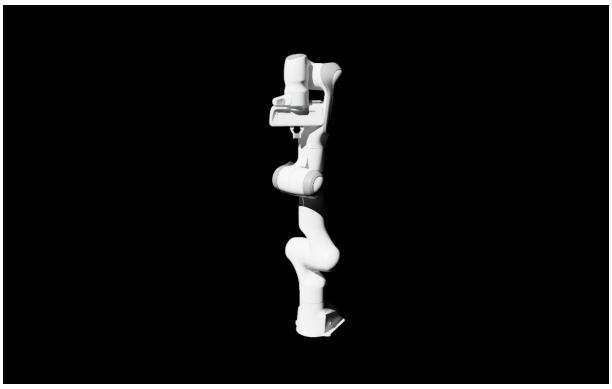

Figure 4.15: Image of the Franka arm USD, with the 7 original joints plus the added fingers.

This robot will serve as the manipulative agent responsible of lifting and placing objects in the desired positions of the environment. Furthermore it will need a consistent controller for articulating all of its joints for a correct coordination, its respective controller development has been mentioned in section 4.4.1.

While directly importing the USD file from the asset explorer does include the necessary collisions and rigid body physics for all of its joints, there is a single inconvenient I found from the default importing and I had to re-configure for a better performance. I got the idea from YouTube tutorial [2]. Basically, as Franka will have to manipulate and lift/place complex shaped objects, it will need capable finger physics,

and the pre-defined finger physics did not perform well enough. The original physics approach of the finger joints was *convexHull*, approach which while being the fastest to calculate and most stable showed inaccuracy when lifting the smallest objects. When trying to make contact with a small object such as a cube, it would collapse with the cube's collision box constantly due to the fingers' big hit boxes, making unfeasible to grasp it correctly. That is why I chose to transition to the *convexDecomposition* physics for the fingers, inspired on the mentioned tutorial. This method decomposes the finger itself into multiple smaller pieces, having each its own convex hit box. This approach worked much better as we were talking about a single and more awkward collision finger compared to a much more sophisticated multiple hit box chain which replicates much better the collision system of a real arm. I got much better results with this technique in the lifting stages of the pipeline.

4.5.1.2 Jetbot - Navigation Agent

The Jetbot configuration was much more friendlier, this agent has no complex joint systems nor articulations of any kind. Even if in image 4.5.1.2 the structure of the robot seems complex, it actually comes up the chassis and the wheels. The chassis acts as the main rigid body of the agent, apart from that I did not need to re-configure the collision physics of the Jetbot. Secondly it has a set of wheels to configure the navigation direction as seen in the Jetbot controller section 4.4.2 and a third one which acts as the balancing wheel which is not related to the engine. The things on top of the basis and those two aerials on the back serve no purpose in the simulator, although they may well have functionality in real cases.

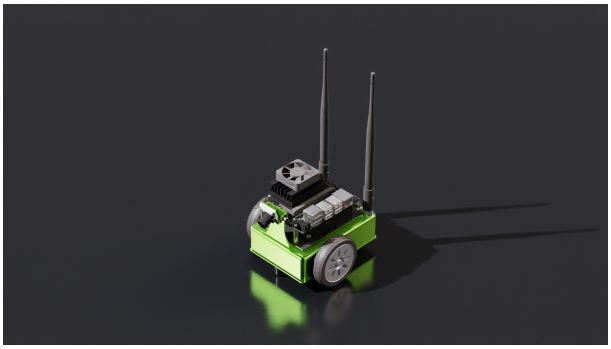

Figure 4.16: Image of the Jetbot arm USD.

4.5.1.3 Cubes

The cubes will be the objects which will be picked and lifted by the robotic arms and placed in the corresponding places. I had doubts on how to create the cubes, whether to load them from the asset

store or create them by myself, and apart from that, there are several ways one can create cubes inside IsaacSim. I finally went for creating them manually for more configuration freedom, and created them using *meshes*. Even if perhaps was not necessary to create them using this class as meshes are made of thousands of little polygons and it would be enough to use a *geometry prim*, I still wanted to experiment with the more complex class and it was personally easier to configure.

Moreover, I made all cubes the same size `[0.05, 0.05, 0.05]`, small enough to be easily manipulable. However, I did experiment with different rotations in more advanced scenarios.

4.5.1.4 Palettes

On the other hand, the palettes will be where the cubes will be placed, they were all loaded from the asset explorer tab, which consists of thousands of objects that can be used in the environments. As seen in figure 4.3.4, I decided to experiment with both different color and shapes. Not only does each platform has a distinct color, but it also has a different hole pattern. Which will make the detection and placement tasks a little bit trickier.

4.5.1.5 Cameras

Cameras are also part of the USD type, they are acquired from camera pins by the use of renderers [3]. As mentioned in earlier stages, I have configured a camera for each point of interest. The cameras will be responsible to capture the screenshot to be sent to the VLM. The main properties of the camera have been repeated in all the environments, which stand as the following:

- **Focal length**: refers to the distance between the optic center of the lens and the image sensor. Inside the simulator, determines what is commonly known as *field of view* (FOV).
- **Focus distance**: defines the distance between the camera and the plane where the objects are. It is used to calculate the *depth of field*, region of the environment where everything which stands out of it will appear blurry. These first two properties were used to calculate the *unprojection* method part of the workflow pipeline.
- **Horizontal aperture**: width length of the camera sensor.
- **Vertical aperture**: height length of the camera.

4.5.1.6 Lights

Light enables a clearer and more realistic view of the scenario, without adding any it would all be black and cameras would not be able to see anything. The light component inside IsaacSim is composed of two attributes. The intensity being a numeric value to represent the strength of the light and the color RGB value [4]. I have kept the default values for both parameters in all my environments.

Apart from that, there are two light configurations inside the simulator. The *direct light* which is the one I have used, shines towards the negative Z-direction, often pointing to the ground from a certain angle. While the second is the *dome light*, which as light beams coming from all directions forming a more uniform lighting. Depending on the configuration chosen, shadows will be formed in one way or another, but I decided to not over-complicate and go for the basic and direct light choice.

4.5.1.7 Physics

All of the scenarios will include physical properties to ensure real verisimilitude. Each object part of the environment will have at least these two physics:

1. **Collision properties**: features which enable the physics engine of the simulator to detect contact between objects. There are several collision configurations such as the ones mentioned at the end of section 4.5.1.1, which differ on the approximation used to calculate the collision among polygons. If there was no collision configuration, objects would trespass each other.

2. **Rigid body**: this other property brings Newton's principles to the objects in the scenario. Applying mass, inertia and gravity laws to them. This property will enable more realistic object behaviors in the physical tasks required in the scenario.

4.5.2 First scenarios

Once I collected all the pieces that would build my different scenarios, I started by developing single agent environments. Individual cases that would then form the final workflow. The very first scenarios consisted of trying out different IsaacSim tutorials, following their official documentation to create the most basic environments, but I do not consider them worthy to mention. Let's jump straight to the interesting scenarios development.

4.5.2.1 First individual Franka environment

This was the first environment where I really put my VLM workflow into test. The setup is basic, a single Franka arm and three different cubes. What I actually wanted to test here was that both my VLM detection and Franka controller worked well. Nevertheless, as this was one of my first experiments, my pipeline was not fully advanced and sophisticated yet. I still had not implemented the computer vision filters, so the VLM purely relied on itself. What I wanted to achieve with this individual environment was the certainty that my pipeline was able to orchestrate the whole VLM and Franka lifting movement, and the environment would end with the robot raising the target cube. This was my workflow at that time, simple yet successful.

1. While running the BentoML server, I still would use the single *ground* method, which had to identify the most likely single target out of the screenshot taken by the single camera in the environment. As a fact, the camera used for this first environment captures the same screenshot seen in image 4.3.4.

2. The VLM, still not as accurate as in the final pipeline, returned the target cube's position and stored it in a JSON file.

3. Finally I load the positions from the storing file and call the *execute_ movement* orchestrator function from the Franka controller (section 4.4.1) to reach on top of the cube, descend and close the grips and finally lift the cube. I decided to decompose the grasping and lifting task into modular functions such as descend, close grips and lift, to gain more control over the whole coordination task and implement more safety techniques.

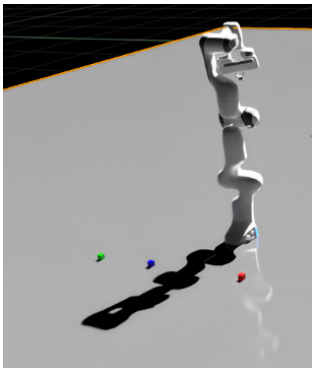

Figure 4.17: First practical scenario, consisting of a single Franka arm and three cubes of different colors. Same size, same orientation.

At this stage of the development, I struggled a lot with the physics engine integrated into the simulator. Such delicate coordination tasks usually returned *warm start* errors, consisting of synchronization issues between the Python interpreter and the physics engine of IsaacSim. The error arose when I would try to run a function when the physics simulation was still running, concept that would lead to total collapse of both the running script and the simulator.

The solution was to perfectly synchronize the python interpreter and the physics renderer, and the best answer I found was the use of the *asyncio* Python library. This library is specialized on asynchronous calls, able to coordinate the calls between the physics engine and python functions. Reason why I decided to inject *async def* functions able to pause the execution using *await* while the other tasks were running. This trick of waiting until the other tasks finished running enabled me to surpass the strict physics engine restrictions and orchestrate a more fluent workflow.

4.5.2.2 Jetbot controller tryout environment

I also crafted a super basic empty environment with just the Jetbot agent to try out the controller (section 4.4.2) I made. It was so simple just containing a basic floor, lighting and the agent, where I would arbitrarily introduce coordinates and test if the robot would travel there successfully. These first two where perhaps the most fundamental environments as would serve as the pillars to the more advanced ones that would come later on, as without these foundations integrating the multi-agent logic would not have been feasible.

4.5.3 Advanced scenarios

After building the foundational environments and sophisticating my final pipeline, it was time to build the multi-agent logic I was looking after. If I were to start straight with multi-agent environment crafting it would have been a much harder task to tackle, but thanks to the previous environments I had a better experience.

4.5.3.1 Multi-agent task workflow

The task the agents must accomplish by cooperation is pretty straightforward: carrying target cube to target pallet. One Franka will receive the task of identifying the target cube's coordinates and will have to lift it and place it on top of the Jetbot, which will be the responsible of traveling between the robotic arms. Once the wheeled agent reaches the second arm, this one will pick the cube on top of the Jetbot

and place it on top of the target platform coordinates. The task may repeat itself, and as there are many cube-platform combinations, I decided to illustrate it with graph 4.18.

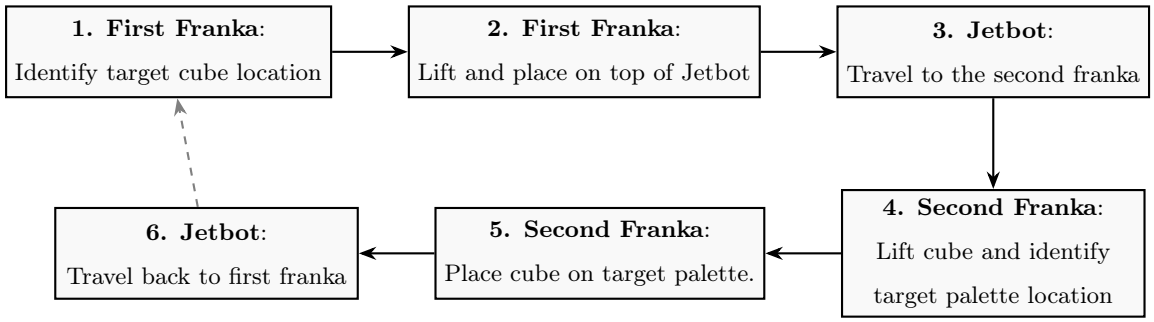

Figure 4.18: Multi-agent cooperative task workflow. The discontinuous line from step 6 to 1 indicates the able to repeat the cycle once again, as the task may consist on transporting more than one cube.

4.5.3.2 Multi-agent scenario

After explaining the workflow the agents of the environment must achieve, I will go over the steps I followed to orchestrate the whole coordination task. The smartest choice in my opinion was to make a main script which would load all the necessary components to carry on the task (see pseudo code 4.5), including the movement scripts and all the target detection material. From that very script I am able to orchestrate the full coordination by just selecting the target cube and target pallet, however, behind that compact structure it all comes up to modular functions.

```
1  await move_first_franka("blue")
2  await place_on_jetbot()
3  await asyncio.sleep(1.0)
4  await move_jetbot(right_json)
5  await pick_from_jetbot("blue")
6  await place_on_palett("black")
7  await move_jetbot(left_json)
```

Pseudocode 4.5: Example of orchestrating call inside the main script, composed by all the modular functions created and introducing the target colors; this call brings the blue cube to the black platform

Let's now go over the procedure hidden beyond those functions, inspired on the workflow graph 4.18. All of the steps carried have been tested on the environment shown in picture 4.5.3.2.

1. **move_first_franka("blue")**: corresponds to the first step node of the graph. Step which consists of detecting the target cube location and grasping the cube, finishing when the arm lifts the cube up to a safe place. At this point of the development the VLM uses the *ground_multi* method. Where among all of the objects appearing in the screenshot sent to the model, it must be able to detect and distinguish the target one. For a fact, image 4.3.4 is the one sent to the model at this step. After successfully detecting the location of the target passing the threshold counter, that position data is saved and loaded for the next stage. The movement phase is decomposed into descending into approaching the object, then descending to a safe contact height, closing the grips and lifting it up to a safe height.

2. **place_on_jetbot**: (step 2 of the graph) once the cube has been lifted is time to place it on top of the Jetbot getting its position. As the Jetbot itself has a complex top surface as seen in image 4.5.1.2, I placed a custom little platform for a better placing performance. That platform has the necessary physics to not fall of when the agent accelerates. The placing movement consists of descending, opening the grips and lifting the arm once again, all of these must take into account not collisioning with the robot's structure, as it would collapse the whole coordination task.

3. **asyncio.sleep(1.0)**: now I make an asynchronous call to relax the python interpreter, this is a security measure to control the synchronization errors.

4. **move_jetbot(right_json)**: corresponding to node 3, I load the right franka arm predicted position captured in screenshot 4.3.4. And travel close enough to the robot to not collapse and facilitate the lifting task, using the custom Jetbot controller functions (4.4.2).

5. **pick_from_jetbot("blue")**: step 4 was definitely the most challenging one, as the placement coordinates of the cube on top of the Jetbot platform was stochastic, and additionally the grasping action of the second arm is the most fragile step of the whole pipeline. Reason why I had to use the ground-truth positions of the cube after being placed on the platform to carry on, as other alternatives I tried would not work as great as I would want to. The lifting movement of this second arm follows the same steps as in step 1, but applying the offset of the cube being on top of the platform.

6. **place_on_jetbot("black")**: in step 5 I load the coordinates got by feeding image 4.3.4 to the

VLM and selecting the target color by the variable introduced when calling the function. As in the other steps, a movement decomposed into little steps is used to finally place the target cube on top of the target pallet.

7. **move_jetbot(left_json)**: even if it not mandatory, it is a great choice to travel back to the other way in case of wanting to try the cycle multiple times.

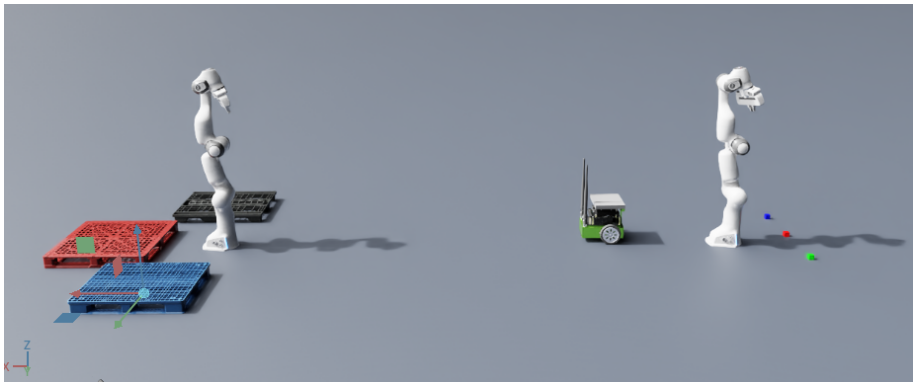

Figure 4.19: First multi agent environment, consisting of two Franka arms, a Jetbot robot and some cubes and platforms.

In case of interest, I put a full practical example of the cycle where all the cubes are transported to all platforms, one by one. It is available at section in the appendix.

4.5.3.3 More advanced alternatives

Even if this project does not pretend to try any *sim2real* transferability, all of these previous environments have shown ideal cases where all of the objects stood at their ideal positions. There were no orientation nor height alternatives for the cubes, meaning that the arm could grab and lift them equally. Reason why I decided to present an alternative version of the environment where the cubes would stand in different configurations, to force the Franka arm to adapt to each target situation.

Figure 4.5.3.3 shows the different alternatives I have prepared for each cube. Where cube red stands as before, the green one is slightly rotated and the blue one stands on top of a custom box. Nevertheless, as the cubes are so tiny compared to the arms fingers open width, the arm is able to grab them independently of their rotation, just needing to configure the box offset to capture successfully the blue one. However, I wanted to introduce these imperfect cases as in real robotic environments not everything falls perfect.

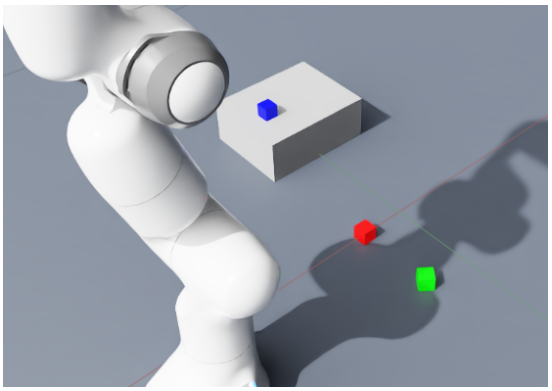

Figure 4.20: Alternative scenario where each cube shows a different position configuration

.

4.6 RL EXPERIMENTS

5. RESULTS AND EVALUATION

5.1 SUCCESSFUL AND FAILURE TRIES

5.2 RESULTS OBTAINED IN EACH SCENARIO

5.3 RESULT DISCUSSION

6. CONCLUSIONS

6.1 CONCLUSIONS AND LESSONS LEARNED

6.2 LIMITATIONS FOUND

The main drawbacks I have found during the development and research process during the whole work have been mainly hardware-related as the specific software requirements necessary to carry on the development are severely demanding due to the fact that the majority of the processes are run and rendered in the GPU. Although I have had access to a good graphical processing unit, the other components of the my working device have fallen behind. Taking for instance the minimum requirements of the used framework *IsaacSim*, I found myself in the limits of the bare minimum, and some of my components such as the CPU were worse than the required. However, I have been able to carry on anyways although there have been a few moments where the I have really felt the bottleneck. When it comes those moments I have felt limited and slowed downed due to my equipment are the following:

- I have needed to split some of the working components of the pipeline process inside *IsaacSim* as the whole process would not fit in my GPU's VRAM. The simple fact of opening the framework already consumed close to a quarter of the graphic card's total capacity, and starting the server to listen and interact with the VLM calls alongside left little to no room for more processes in the background. The fact of working on the edge of the minimum hardware requirements also slowed the rendering of visual work and reduced the number of frames per second, nevertheless, this did not obstruct the development process.

- When it comes to making the most of the reinforcement learning training process, I have needed to tweak some of the predefined system configurations to maximize the performance and reduce any kind of lagging and overflow. Adapting the GPU capacity properties to maximize the overall functioning. If I were to work with better equipment, I would have liked to go for longer and deeper trainings, however, I am happy with the performance I obtained when tweaking the parameters I will have discussed in the corresponding section and other strategies such as parallelization.

- Not all limitations have been hardware related, as the official NVIDIA documentation websites were usually corrupted depending on the version. Leading to consulting external pages to solve the mismatching dependencies or installation processes for the frameworks used.

- After consulting the official documentation of the *IsaacLab* repository, I found that even the default reinforcement learning training examples and demonstrations did not work as intended independently of the version. I was not the only one with the same issues as other peoples raised the same issues in different forums, so I inspired in the solutions given by the users with the same problems.

6.3 FUTURE WORK

BIBLIOGRAPHY

[1]

[2]

[3]

[4]

[5] Multimodal learning. https://en.wikipedia.org/wiki/Multimodal_learning. Accessed: April 23, 2026.

[6] Edge AI and Vision Alliance. Vision as a service: Democratization of vision for consumers and businesses (a presentation from tend), September 2015. Accessed: 2024-05-22.

[7] BentoML Team. Bentoml: The unified model serving framework.

[8] EAGERx Contributors. Eagerx: Engine agnostic graph-based robotic experience. https://github.com/eager-dev/eagerx, 2023.

[9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.

[10] EleventhHourEnthusiast. Llava and llava-1.5: Visual instruction tuning. https://medium.com/@EleventhHourEnthusiast/llava-and-llava-1-5-1cf8be377245, 2023. Accessed: 2026-04-24.

[11] Gravio Team. Vision-language models (VLM) vs visual question answering (VQA) in 2025? https://www.gravio.com/en-blog/vision-language-models-vlm-vs-visual-question-answering-vqa-in-2025, Mar 2025. Gravio Blog, Accessed: [Insert Date Here].

[12] Jack Hessel, Ari Holtzman, Maxwell Forbes, Ronan Le Bras, and Yejin Choi. Clipscore: A reference-free evaluation metric for image captioning, 2022.

[13] Zilin Huang, Zhengyang Wan, Zihao Sheng, Boyue Wang, Junwei You, Yue Leng, and Sikai Chen. Sim2real-ad: A modular sim-to-real framework for deploying vlm-guided reinforcement learning in real-world autonomous driving, 2026.

[14] Andrew Jaegle, Felix Gimeno, Andrew Brock, Andrew Zisserman, Oriol Vinyals, and Joao Carreira. Perceiver: General perception with iterative attention, 2021.

[15] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model, 2024.

[16] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning (ICML)*. PMLR, 2023.

[17] Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, Octi Zhang, Peter Du, Antonio Serrano-Muñoz, Xinjie Yao, René Zurbrügg, Nikita Rudin, Lukasz Wawrzyniak, Milad Rakhsha, Alain Denzler, Eric Heiden, Ales Borovicka, Ossama Ahmed, Iretiayo Akinola, Abrar Anwar, Mark T. Carlson, Ji Yuan Feng, Animesh Garg, Renato Gasoto, Lionel Gulich, Yijie Guo, M. Gussert, Alex Hansen, Mihir Kulkarni, Chenran Li, Wei Liu, Viktor Makoviychuk, Grzegorz Malczyk, Hammad Mazhar, Masoud Moghani, Adithyavairavan Murali, Michael Noseworthy, Alexander Poddubny, Nathan Ratliff, Welf Rehberg, Clemens Schwarke, Ritvik Singh, James Latham Smith, Bingjie Tang, Ruchik Thaker, Matthew Trepte, Karl Van Wyk, Fangzhou Yu, Alex Millane, Vikram Ramasamy, Remo Steiner, Sangeeta Subramanian, Clemens Volk, CY Chen, Neel Jawale, Ashwin Varghese Kuruttukulam, Michael A. Lin, Ajay Mandlekar, Karsten Patzwaldt, John Welsh, Huihua Zhao, Fatima Anes, Jean-Francois Lafleche, Nicolas Moënne-Loccoz, Soowan Park, Rob Stepinski, Dirk Van Gelder, Chris Amevor, Jan Carius, Jumyung Chang, Anka He Chen, Pablo de Heras Ciechomski, Gilles Daviet, Mohammad Mohajerani, Julia von Muralt, Viktor Reutskyy, Michael Sauter, Simon Schirm, Eric L. Shi, Pierre Terdiman, Kenny Vilella, Tobias Widmer, Gordon Yeoman, Tiffany Chen, Sergey Grizan, Cathy Li, Lotus Li, Connor Smith, Rafael Wiltz, Kostas Alexis, Yan Chang, David Chu, Linxi "Jim" Fan, Farbod Farshidian, Ankur Handa, Spencer Huang, Marco Hutter, Yashraj Narang, Soha Pouya, Shiwei Sheng, Yuke Zhu, Miles Macklin, Adam Moravanszky, Philipp Reist, Yunrong Guo, David Hoeller, and Gavriel State. Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025.

[18] Nikon USA. Understanding focal length, 2024.

[19] NVIDIA.

[20] NVIDIA Corporation. *RMPflow: Reactive Motion Policies for Robot Control.* NVIDIA Isaac Sim Documentation, 2024. Accessed: 2024-05-22.

[21] OpenAI. Clip: Contrastive language-image pre-training. https://github.com/openai/CLIP, 2021. Original implementation of the CLIP model.

[22] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021.

[23] Nils Rethmeier and Isabelle Augenstein. A primer on contrastive pretraining in language processing: Methods, lessons learned and perspectives, 2021.

[24] RoboDK.

[25] Kyle Simek. Dissecting the camera matrix, part 2: The extrinsic matrix, August 2012. Accessed: 2024-05-22.

[26] Jianlin Su. Rope 2d: Rotary position embedding for 2d, May 2021. Scientific Spaces.

[27] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.

[28] Haochen Tan, Wei Shao, Han Wu, Ke Yang, and Linqi Song. A sentence is worth 128 pseudo tokens: A semantic-aware contrastive learning framework for sentence embeddings, 2022.

[29] Chen Tang, Ben Abbatematteo, Jiaheng Hu, Rohan Chandra, Roberto Martín-Martín, and Peter Stone. Deep reinforcement learning for robotics: A survey of real-world successes, 2024.

[30] M. Timothy. What is phrase grounding? https://blog.roboflow.com/what-is-phrase-grounding/, Nov 2024. Roboflow Blog, Accessed: [Insert Date Here].

[31] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.

[32] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution, 2024.

[33] Zirui Wang, Jiahui Yu, Adams Wei Yu, Zihang Dai, Yulia Tsvetkov, and Yuan Cao. Simvlm: Simple visual language model pretraining with weak supervision, 2022.

[34] Wikipedia. Cross-modal retrieval. https://en.wikipedia.org/wiki/Cross-modal_retrieval, 2026. Accessed: 2026-04-23.

[35] Wikipedia contributors. Bicubic interpolation — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Bicubic_interpolation, 2024. [Online; accessed 19-April-2026].

[36] Wikipedia contributors. Contrastive language-image pre-training — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Contrastive_Language-Image_Pre-training, 2024. [Online; accessed 19-April-2026].

[37] Wikipedia contributors. Knowledge distillation — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Knowledge_distillation, 2024. [Online; accessed 20-April-2026].

[38] Wikipedia contributors. Manifold alignment — Wikipedia, the free encyclopedia, 2024. [Online; accessed 22-April-2026].

[39] Wikipedia contributors. Vision-language-action model — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Vision-language-action_model, 2026. [Online; accessed 24-April-2026].

[40] Wikipedia contributors. Vision-language model — Wikipedia, the free encyclopedia, 2026.

[41] Xiefeng Wu, Jing Zhao, Shu Zhang, and Mingyu Hu. Teaching rl agents to act better: Vlm as action advisor for online reinforcement learning, 2025.

[42] Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of ppo in cooperative, multi-agent games, 2022.

[43] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, Cong Wei, Botao Yu, Ruibin Yuan, Renliang Sun, Ming Yin,

Boyuan Zheng, Zhenzhu Yang, Yibo Liu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi, 2024.

[44] Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. Multi-agent reinforcement learning: A selective overview of theories and algorithms, 2021.

7. APPENDIX

7.1 INSTALATION GUIDE

IsaacSim/Lab

7.2 RELEVANT CODE

7.2.1 Helping offsets used for VLM guidance

```
OFFSETS = {
  "cube": {
    "red": [0.070, -0.025, 0.0],
    "green": [0.222, 0.133, 0.0],
    "blue": [0.004, -0.005, 0.0],
  },
  "pallet": {
    "red": [0.427, -0.295, 0.0],
    "blue": [0.220, 1.025, 0.0],
    "black": [-0.084, 0.294, 0.0],
  },
  "robot": {"left": [0.56, -0.39, 0.0], "right": [-1.62, -0.93, 0.0]},
}
```

Pseudocode 7.1: Calibration offsets used, (x, y, z) triplets where for all cases z=0 (objects standing on top of the ground)

7.2.2 Full multi-agent coordination script

```
async def main():
print("##### STARTING FULL PIPELINE (LIFT+PLACE+NAVIGATION) STILL WITH
    CONTROLLER #####")

# first cube interaction
```

```
    await move_first_franka("red")
    await place_on_jetbot()
    await asyncio.sleep(1.0)
    await move_jetbot(right_json)
    await pick_from_jetbot("red")
    await place_on_palett("red")
    await move_jetbot(left_json)

    # second cube
    await move_first_franka("green")
    await place_on_jetbot()
    await asyncio.sleep(1.0)
    await move_jetbot(right_json)
    await pick_from_jetbot("green")
    await place_on_palett("blue")
    await move_jetbot(left_json)

    # third cube
    await move_first_franka("blue")
    await place_on_jetbot()
    await asyncio.sleep(1.0)
    await move_jetbot(right_json)
    await pick_from_jetbot("blue")
    await place_on_palett("black")
    await move_jetbot(left_json)
    print("##### PIPELINE FINISHED! #####")


    asyncio.ensure_future(main())
```

Pseudocode 7.2: After seeing a complete cube transporting cycle, this script involves the full cycle involving all cubes and platforms

7.3 PROMPTS USED

7.3.1 Helping prompts used for the VLM target detection task

```
CUBE_PROMPTS = {
  "red": ["red cube in the center", "small red block near robot"],
  "green": ["small green cube on the right side", "green block on the
     right"],
  "blue": ["small blue cube on the left side", "blue block on the left"],
}

PALLET_PROMPTS = {
  "red": [
  "red plastic pallet on the floor",
  "top view of a red cargo pallet near the robot",
  "the center of the red rectangular grid structure",
  ],
  "blue": [
  "blue plastic pallet located to the left",
  "blue industrial pallet on the grey surface",
  "top-down view of a blue rectangular pallet",
  ],
  "black": [
  "black plastic pallet on the right side",
  "dark grey cargo pallet on the floor",
  "rectangular black grid structure",
  ],
}

ROBOT_PROMPTS = {
  "white": [
  "white robotic arm located at the far left edge of the image",
  "white robotic arm located at the far right edge of the image",
  "franka emika panda arm near the corners",
  "robotic manipulator, ignore the mobile platform in the center",
  ]
}
```

Pseudocode 7.3: Custom prompts used for ground_multi detection function to try out different target names and then take the median of the predicted coordinates.

7.4 RESULT TABLE

7.5 TENSORBOARD ANALYSIS